

Quantum One-Time Programs: Less Assumptions, More Feasibility, and One-Message 2PC

Prabhanjan Ananth*
UCSB

Divyanshu Bhardwaj†
UCSB

Abstract

Quantum one-time programs, introduced by Broadbent, Gutowski, and Stebila [CRYPTO 2013], is a compiler that takes as input a program f and converts it into a quantum state σ_f such that given σ_f , it is possible to only obtain the output of f on a single input. Our goal is to investigate the class of functionalities for which we can design one-time programs in the plain model. We show the following:

- We show that, assuming learning with errors, one-time programs for (randomized) pseudorandom functionalities exist in the plain model. In contrast, the result by Gupte, Liu, Raizes, Roberts, and Vaikuntanathan [STOC 2025] relied upon indistinguishability obfuscation.
- We show that, assuming indistinguishability obfuscation, one-time programs for any class of randomized functionalities exist in the plain model, as long as the class of functionalities satisfy a natural security notion called constrained security.

Additionally, we give constructions of one-time encryption and one-time message authentication codes based on learning with errors. Finally, we show that a novel variant of one-time programs imply one-message secure two-party computation protocols for non-trivial functionalities in the plain model.

*prabhanjan@cs.ucsb.edu

†dbhardwaj@ucsb.edu

Contents

1	Introduction	4
1.1	Results	5
1.2	Positive Results	6
1.2.1	Applications	6
2	Technical Overview	7
2.1	QOTP for Privately Constrained Functions	7
2.2	One-Time Programs for Constrained Functionalities	10
2.3	Malicious QOTP	11
2.3.1	Implication to One-Message 2PC	12
2.4	Related Work	13
3	Preliminaries	13
3.1	Notation	13
3.2	Privately Constrained Pseudorandom Functions	13
3.3	Indistinguishability Obfuscation	15
3.4	Subspace Hiding Obfuscator	15
3.5	Non Interactive Zero Knowledge Proof of Knowledge	16
3.6	Trace Distance	16
3.7	Direct Product Hardness of Coset States	17
4	Definitions	18
4.1	Quantum One-Time Programs	18
4.2	Constrained Functionalities	19
4.3	Privately Constrained Functionality	21
5	QOTP for Privately Constrained Functionality	23
5.1	Construction	23
5.2	Correctness and Security	24
5.3	QOTP for Pseudorandom Functions	30
6	QOTP for Constrained Functionalities	31
6.1	Construction	31
6.2	Correctness and Security	32
7	Malicious Security	34
7.1	Construction	37
7.2	Correctness and Security	38
8	Applications	42
8.1	Secret Key Encryption	42
8.1.1	Preliminaries	42
8.1.2	QOTP for Secret Key Encryption	43
8.2	Message Authentication Code	45
8.2.1	Preliminaries	45

8.2.2	QOTP for Message Authentication Code	45
8.3	One-Message 2PC	47

1 Introduction

One-time program is a seminal notion in cryptography introduced by Goldwasser, Kalai, and Rothblum [GKR08]. A one-time program is a compiler that takes as input a program f and turns into another program $\hat{f}$ that efficiently implements f . The security guarantee states that using $\hat{f}$, one can only obtain the output of f on a single input. If $\hat{f}$ is represented as a classical string then this security definition is clearly impossible to achieve. To circumvent this impossibility, one has to resort to either using trusted hardware, as suggested by [GKR08], or resort to other computational models such as quantum computing. The many recent leakage attacks on trusted hardware [KHF+20] have brought into scrutiny the use of trusted hardware in cryptography design.

In a pioneering work, Broadbent, Gutoski and Stebila [BGS13], proposed using quantum resources to design one-time programs. Concretely, in a quantum one-time program, a program f is compiled into a quantum state ρ_f such that given ρ_f , we can compute $f(x)$ only on a single input x . While [BGS13] showed that quantum one-time programs for deterministic functionalities is impossible, a couple of recent exciting works initiated by Gupte, Liu, Raizes, Roberts and, Vaikuntanathan [GLR+24] and by Gunn and Movassagh [GM24] provide the first feasibility results of quantum one-time programs for randomized functionalities:

- In the classical oracle model, both works [GM24; GLR+24] present constructions of quantum one-time programs for a class of randomized functionalities. In addition, [GLR+24] showed that not all classes of randomized functionalities can be realized in the plain model.
- In the plain model, [GLR+24] present a construction of quantum one-time programs for pseudorandom functions based on the assumption of post-quantum secure indistinguishability obfuscation.
- [GLR+24] showed that one-time programs for publicly-verifiable randomized functionalities imply public-key quantum money [Wie83; Aar09; Zha21], a foundational primitive in quantum cryptography. Roughly speaking, a class of functionalities $\mathcal{F}$ is publicly verifiable if it is equipped with a public verification algorithm such that the output of any function $f \in \mathcal{F}$ passes the verification. As an example, signing functionalities are publicly-verifiable. In the same work, they also presented constructions of one-time proofs and signature tokens.

The compelling applications of one-time programs prompt us to better understand the feasibility landscape of quantum one-time programs in the plain model. As mentioned above, [GLR+24] already ruled out the possibility of one-time programs for all randomized functionalities in the plain model. In light of this, we ask the following question:

Q1. What are the class of functionalities for which we can design quantum one-time programs in the plain model?

The necessity of post-quantum indistinguishability obfuscation in the construction of one-time programs for randomized pseudorandom functions by [GLR+24] is unclear. Given the implication of one-time programs for *publicly verifiable* functionalities to public-key quantum money, realizing one-time programs for these functionalities would at least require the same cryptographic assumptions under which public-key quantum money exist. The only known constructions of public-key quantum money are based on indistinguishability obfuscation [Zha21], group actions [Zha23] or novel cryptographic assumptions that are yet to be scrutinized [KSS21; LMZ23]. On the other hand, pseudorandom functionalities are not publicly-verifiable. This raises the following question:

Q2. Can we base the existence of quantum one-time programs for pseudorandom functions on weaker assumptions?

Another direction that needs further exploration is the implications of quantum one-time programs to secure multi-party computation (MPC). While hardware-based one-time programs have been used to design low round secure multiparty computation [GIS+10], it has not been known whether quantum one-time programs are similarly useful for designing low round MPC. We also explore this direction in this work.

1.1 Results

We address both of the above questions *Q1* and *Q2*. In addition, we also explore the applications of quantum one-time programs.

Functionalities of Interest. We first focus on functionalities that satisfy constrained security. A class of keyed functionalities¹ $\text{CF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$ can be constrained if it is associated with two efficient algorithms:

- Constrain algorithm, **Constrain**: it takes as input a key k , program $P : \{0, 1\}^{n+r} \rightarrow \{0, 1\}$ and outputs a constrained key k_P .
- Evaluation algorithm, **CEval**: it takes as input a constrained key k_P , function inputs $x \in \{0, 1\}^n$, $u \in \{0, 1\}^r$ and outputs $y \in \{0, 1\}^m$.

Since we are interested in *randomized* constrained functionalities, we will henceforth treat u as a random string. On any input (x, u) such that $P(x, u) = 1$, we have the correctness guarantee that $\text{CEval}(k_P, x) = f_k(x; u)$.

In terms of security, there are two types of security we desire:

- *Search (constrained) security*: given k_P , it is computationally hard to predict the output of $f_k(x; u)$ when $P(x, u) = 0$.
- *Decision (constrained) security*: this is parameterized by a distribution $\mathcal{D}$ such that given k_P , it is computationally hard to distinguish the output of $f_k(x; u)$ versus $\mathcal{D}$.

In both the security definitions above, the adversary is allowed to obtain many outputs $\{f_k(x_i; u_i)\}$ for (x_i, u_i) of its choice as long as $(x_i, u_i) \neq (x, u)$. Specific instantiations of constrained cryptographic functionalities, such as pseudorandom functions [BW13; BGI14; KPTZ13; BV15; BLW17; HKKW19], signatures [BGI14; BZ17] and encryption schemes [CHJV14; Wat15; AJJM22; AKN24] have been extensively studied in the literature.

We also consider stronger security guarantees wherein the constraint P is hidden from the description of k_P . We term the class of functionalities satisfying this property as *privately constrained* functionalities. Concretely, we require that k_{P_0} and k_{P_1} are computationally indistinguishable, where P_0 and P_1 are two constraint programs. In addition, the adversary is allowed to obtain outputs of the form $f_k(x; u)$ for any input (x, u) of its choice as long as $P_0(x, u) = P_1(x, u)$. We call this constrained hiding property². A popular special case of this notion is private constrained pseudorandom functions which can be instantiated from learning with errors [BTVW17; PS18].

¹Any class of functionalities can be efficiently represented as keyed functionalities using universal circuits.

²Instead of defining constrained hiding separately, in the technical sections, we integrate constrained hiding security directly into the search and decisional security definitions for privately constrained functionalities.

1.2 Positive Results

We present constructions of quantum one-time programs for both constrained and privately constrained functionalities.

Theorem 1 (Informal). *There exists a quantum one-time program for private constrained functionalities.*

We emphasize that the above result does not make any additional computational assumptions. If we instantiate the above theorem for privately constrained pseudorandom functions, which can be based on learning with errors [BTVW17; PS18], we obtain the following result.

Corollary 2. *Assuming that learning with errors is hard against quantum polynomial time algorithms, there exists a quantum one-time program for pseudorandom functions.*

This improves upon the work of Gupte et al. [GLR+24] who showed the same result additionally assuming the existence of post-quantum secure indistinguishability obfuscation. At the heart of our result is proving a computational direct product hardness property of coset states based on private constrained functionalities; the prior works either showed information-theoretic direct product hardness [BS23; CLLZ21; CHV23] or computational (iO-based) direct product hardness [CLLZ21; CHV23].

The security definition of quantum one-time programs that we consider needs further clarification: as in [GLR+24], we consider the definition of *operational security* which states that given a quantum one-time program of a function f_k , it should be infeasible to produce two distinct input-randomness pairs (x_1, u_1) and (x_2, u_2) along with their outputs (y_1, y_2) such that $f_k(x_1; u_1) = y_1$ and $f_k(x_2; u_2) = y_2$. We call this *search* operational security. We also define *decisional* operational security wherein the adversary cannot distinguish either the output $f_k(x_1; u_1)$ versus $\mathcal{D}$ or the output of $f_k(x_2; u_2)$ versus $\mathcal{D}$, for some distribution³ $\mathcal{D}$. As far as we know, the only known feasibility result of quantum one-time programs, which is for pseudorandom functionalities, also only satisfies operational security.

Theorem 3 (Informal). *Assuming the existence of post-quantum secure indistinguishability obfuscation, there exist quantum one-time programs for constrained functionalities.*

One might wonder whether the use of indistinguishability obfuscation in the above theorem is inherent. We argue that constructing quantum one-time programs for constrained functionalities without additional assumptions would result in major progress on public-key quantum money. Indeed, an unconditional result would imply public-key quantum money from learning with errors: the reason being that we can instantiate constrained signatures (a special class of constrained functionalities) from learning with errors and moreover, one-time program for constrained signatures imply public-key quantum money.

1.2.1 Applications

We present several applications.

³As an example, $\mathcal{D}$ is the uniform distribution in the context of pseudorandom functions.

One-Message Secure Two-Party Computation. We consider the well studied problem of secure two-party computation. Since there are several settings concerning trust models, round complexity constraints and adversarial models, we will first clarify the precise setting we consider in this work.

- There are two parties P_1 and P_2 . The party P_1 's input is referred to as x_1 and P_2 's input is referred to as x_2 . Denote the functionality to be securely computed to be g .
- P_1 only sends a single message to P_2 . Additionally, both parties have access to a common reference string (CRS). At the end of the computation, P_2 receives the output. If both P_1 and P_2 behaved honestly, the output is $g(x_1, x_2; r)$ for some randomness r .
- We require simulation security against P_1 while we require operational security against P_2 . We defer the formal definitions to the technical sections.

We show the following.

Theorem 4 (Informal). *There exists a two-party (randomized) functionality $\mathcal{F}$ for which there exists a two-party secure computation for $\mathcal{F}$ assuming the existence of post-quantum secure indistinguishability obfuscation and learning with errors.*

In particular, we consider $\mathcal{F}$ for which one-message two-party secure computation is impossible with purely classical resources. At a high level, suppose $\text{CF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$ is a class of constrained functionalities. Corresponding to this, we can define $\mathcal{F}$ as follows: it takes as input (k, x) , samples randomness r and outputs $f_k(x; u)$, where u is a function of (k, x, r) .

It is important to note that it is impossible to achieve one-message secure two-party computation for all (even randomized) functionalities using quantum resources as observed by [BGS13; GLR+24]. Thus, we can only hope to design two-party computation for specialized functionalities.

Applications of Quantum One-Time Pseudorandom Functions. We consider two applications of one-time pseudorandom functions.

First, we consider a notion of one-time secret key encryption, where it is possible to issue a one-time encryption key that lets a user generate just one ciphertext. This notion can be viewed as an intermediate notion between secret-key encryption and public-key encryption. In the traditional secret key encryption scheme, anyone who does not possess the secret key cannot encrypt whereas in the public key encryption scheme, anyone can encrypt.

Secondly, we consider the notion of one-time message authentication codes (MAC) wherein a one-time MAC key can be issued in such a way that using this key, one can generate a MAC only on one message and nothing else. We show the following:

Theorem 5 (Informal). *Assuming quantum one-time programs for (randomized) pseudorandom functions, there exist one-time secret-key encryption and one-time message authentication codes.*

2 Technical Overview

2.1 QOTP for Privately Constrained Functions

We discuss the main ideas behind Theorem 1. Suppose we have a class of private constrained functionalities $\text{PCF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$. We will assume that PCF satisfies

decisional security. The same template can also be adapted for the case when PCF satisfies *search* security.

To compute a one-time program for a function f_k , do the following:

- First generate n coset states, where the i^{th} coset state is defined as follows:

$$\left| A_{s_i, s'_i}^{(i)} \right\rangle = \frac{1}{\sqrt{|A_i|}} \sum_{a_i \in A_i} (-1)^{\langle a_i, s'_i \rangle} |a_i + s_i\rangle$$

Here, A_i is a random subspace of $\mathbb{F}_2^\lambda$ of dimension $\lfloor \frac{\lambda}{2} \rfloor$. Moreover, $s_i, s'_i \xleftarrow{\$} \mathbb{F}_2^\lambda$.

- Next constrain the function f_k using the constraint circuit $P_{\mathbf{A}}$ to obtain $k_{\mathbf{A}}$. Informally speaking, the constraint $P_{\mathbf{A}}$ takes as input x along with n vectors $u^{(1)}, \dots, u^{(n)}$. It performs the following check for every i : $x^{(i)} = 0$ if and only if $u^{(i)} \in A_i + s_i$ or $x^{(i)} = 1$ if and only if $u^{(i)} \in A_i^\perp + s'_i$. If even one check is violated then it returns 0, otherwise it returns 1.

Set $\left(\left\{ \left| A_{s_i, s'_i}^{(i)} \right\rangle \right\}_i, k_{\mathbf{A}} \right)$ to be the one-time program computing f_k .

To evaluate on an input x , we measure the coset states to obtain the vectors $\mathbf{u} = (u^{(1)}, \dots, u^{(n)})$ such that $u^{(i)} \in A_i + s_i$ if $x^{(i)} = 0$ and $u^{(i)} \in A_i^\perp + s'_i$ if $x^{(i)} = 1$. Evaluate the constrained function $k_{\mathbf{A}}$ on $(x, u^{(1)}, \dots, u^{(n)})$ to obtain y . Since the constraint $P_{\mathbf{A}}(x, u^{(1)}, \dots, u^{(n)}) = 1$ by definition, we have that $y = f_k(x; \mathbf{u})$.

Security: Challenges. To prove security, suppose there exists a quantum polynomial time (QPT) adversary that breaks decisional weak operational security. That is, for two input-randomness pairs $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$, it can distinguish (with non-negligible advantage) both $f_k(x_1; \mathbf{u}_1)$ versus a sample from $\mathcal{D}$ and $f_k(x_2; \mathbf{u}_2)$ versus a sample from $\mathcal{D}$.

Let us first discuss the main challenges we encounter before diving into the details. There are two possibilities.

Case 1: Both inputs satisfy $P_{\mathbf{A}}$. The first possibility is that the constraint program $P_{\mathbf{A}}(x_i, \mathbf{u}_i)$, for both $i \in \{1, 2\}$, outputs 1, i.e., both the inputs satisfy the constraint. In this case, we first observe that by design if $(x_1, \mathbf{u}_1) \neq (x_2, \mathbf{u}_2)$ then $\mathbf{u}_1 \neq \mathbf{u}_2$. Let us first discuss the main challenges we encounter before diving into the details. There are two possibilities. In this case, our hope is to prove that we violate the direct product hardness of coset states. There are two flavors of direct product hardness studied in the literature [CLLZ21; CHV23]:

- Information-theoretic direct product hardness: given one copy of a coset state $\frac{1}{\sqrt{|A|}} \sum_{a \in A} (-1)^{\langle a, s' \rangle} |a + s\rangle$ and given quantum query access to oracles that test membership in $A + s$ and $A^\perp + s'$, except with negligible probability, cannot produce u, v such that: (a) $u \in A + s, v \in A^\perp + s'$, (b) $u, v \in A + s$ where $u \neq v$ and, (c) $u, v \in A^\perp + s'$ where $u \neq v$. There is another variant considered by [CHV23] where the adversary, instead of having access to oracles, gets as input the descriptions of T, V, t, t' where T is a random superspace of A (of dimension $\lfloor \frac{3\lambda}{4} \rfloor$) and V is a random subspace of A (of dimension $\lfloor \frac{\lambda}{4} \rfloor$). Moreover, t, t' are random vectors subject to the constraint that $A + s \subseteq T + t$ and $A^\perp + s' \subseteq V^\perp + t'$.

- Computational (iO-based) direct-product hardness: given one copy of a coset state $\frac{1}{\sqrt{|A|}} \sum_{a \in A} (-1)^{\langle a, s' \rangle} |a + s\rangle$ and given $\hat{A}, \hat{A}^\perp$ (where $\hat{A}$ is the obfuscation of the program checking membership in $A + s$, $\hat{A}^\perp$ is the obfuscation of the program checking membership in $A^\perp + s'$), except with negligible probability, cannot (in quantum polynomial-time) produce u, v such that: (a) $u \in A + s, v \in A^\perp + s'$, (b) $u, v \in A + s$ where $u \neq v$ and, (c) $u, v \in A^\perp + s'$ where $u \neq v$.

Unfortunately both flavors are not immediately useful for us! We cannot work in the information-theoretic setting because the adversary gets information about the subspaces and the coset representatives embedded in the constraint program $P_{\mathbf{A}}$. At the same time we cannot use the iO-based direct product hardness property because we would like to avoid the use of obfuscation. We propose the design of a third flavor of computational direct-product hardness property that is based on private-constrained functionalities; we discuss this later.

Case 2: At least one of the inputs do not satisfy $P_{\mathbf{A}}$. The second possibility is that the constraint program $P_{\mathbf{A}}(x_i, \mathbf{u}_i)$, for some $i \in \{1, 2\}$, outputs 0, i.e., at least one of the inputs does not satisfy the constraint. In this case, the hope is to violate the security of the private-constrained functionality. A natural idea is to consider a constraint program P_{null} such that P_{null} is an all zeroes function. That is, on every $(x, \mathbf{u})$, we have $P_{\text{null}}(x, \mathbf{u}) = 0$. Then, the idea would be to switch $k_{\mathbf{A}}$ (as computed in the description of the one-time program) with k_{null} , where k_{null} is obtained by constraining f_k with P_{null} . Unfortunately, we do not know how to perform reduction in this case. Informally speaking, the issue is that in order to successfully utilize the one-time program adversary, reduction will be forced to make queries (to the challenger of private constrained security) of the form (x_i, u_i) where $P_{\mathbf{A}}(x_i, \mathbf{u}_i) = 1$ but $P_{\text{null}}(x_i, \mathbf{u}_i) = 0$. But such queries are disallowed in the security experiment of private constrained functionalities; refer to Definition 28 for more details.

We show how to come up with a different constraint program $P_{\mathbf{T}, \mathbf{V}}$ (we will discuss the description later) such that the constrained key $k_{\mathbf{A}}$ can be safely switched to $k_{\mathbf{T}, \mathbf{V}}$ (obtaining by constraining f_k with $P_{\mathbf{T}, \mathbf{V}}$) while ensuring that the reduction is admissible; i.e., it only makes allowed queries to the external challenger.

Proof Template. We present the proof template below.

Hybrid 0: This corresponds to the real world. The adversary gets $\left(\left\{ \left| A_{s_i, s'_i}^{(i)} \right\rangle \right\}_i, k_{\mathbf{A}} \right)$ from the challenger. The adversary then submits two input-randomness pairs $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$. The challenger then samples two uniformly random bits b_1, b_2 . It then sends $f_k(x_1; \mathbf{u}_1)$ (if $b_1 = 0$) or a sample from $\mathcal{D}$ (if $b_1 = 1$). Similarly, it sends $f_k(x_2; \mathbf{u}_2)$ (if $b_2 = 0$) or a sample from $\mathcal{D}$ (if $b_2 = 1$). The adversary then outputs (b'_1, b'_2) and it wins if $b'_i = b_i$.

Hybrid 1: The only difference with the previous hybrid is that the challenger generates the following program $P_{\mathbf{T}, \mathbf{V}}$ which is defined as follows: the program $P_{\mathbf{T}, \mathbf{V}}$ takes as input $(x = x^{(1)} \dots x^{(n)}, \mathbf{u} = (u^{(1)}, \dots, u^{(n)}))$,

- Parse $\mathbf{T}$ as $\{T_i + t_i\}_{i \in [n]}$ and $\mathbf{V}$ as $\{V_i^\perp + t'_i\}_{i \in [n]}$, where $V_i \subseteq A_i \subseteq T_i$. In more detail V_i is a random subspace of A_i of dimension $\lfloor \frac{\lambda}{4} \rfloor$ and T_i is a random superspace of A_i of dimension $\lfloor \frac{3\lambda}{4} \rfloor$.

- First it checks if either of the following holds: (a) $x^{(i)} = 0$ and $u^{(i)} \in T_i + t_i$ or, (b) $x^{(i)} = 1$ and $u^{(i)} \in V_i^\perp + t'_i$.
- If all the checks are satisfied then output 1. Else, output 0.

We emphasize that the challenger does not use the above program in any way to respond to the adversary. Clearly, this hybrid is identical to the previous one.

Hybrid 2: this is similar to the previous hybrid except in the generation of the constrained key. Previously, we constrained f_k using $P_{\mathbf{A}}$. We now constrain f_k using $P_{\mathbf{T}, \mathbf{V}}$.

The high level idea is to invoke the security of private constrained functionalities to switch from Hybrid 1 to Hybrid 2. Once we are in Hybrid 2, we can then invoke the direct product hardness of coset states, specifically the variant by [CHV23], wherein the descriptions of $\mathbf{T}, \mathbf{V}$ are made public.

Let us first focus on switching from Hybrid 1 to Hybrid 2. As we discussed in *Case 2*, in order to invoke the private constrained security, we need to ensure that the reduction never needs to answer queries that are satisfied by $P_{\mathbf{T}, \mathbf{V}}$ but not by $P_{\mathbf{A}}$. We show this in Lemma 37 and Lemma 38. At a high level, we first argue (refer to Lemma 37) that the probability that the adversary makes such queries in Hybrid 1 is negligible. This is because $\mathbf{T}, \mathbf{V}$ is information-theoretically hidden from the adversary in Hybrid 1. Similarly, we argue (refer to Lemma 38) that the probability that the adversary makes such queries in Hybrid 2 is also negligible. If not, there is a reduction that uses the programs $P_{\mathbf{T}, \mathbf{V}}$ and $P_{\mathbf{A}}$ to break the security of private constrained functionalities.

Once we are in Hybrid 2, we consider two cases:

- Both $P_{\mathbf{A}}(x_1, \mathbf{u}_1) = 1$ and $P_{\mathbf{A}}(x_2, \mathbf{u}_2) = 1$. Suppose $\mathbf{u}_1 = (u_1^{(1)}, \dots, u_1^{(n)})$ and $\mathbf{u}_2 = (u_2^{(1)}, \dots, u_2^{(n)})$. In this case, since $\mathbf{u}_1 \neq \mathbf{u}_2$, we have that there exists $i \in [n]$ such that $u_1^{(i)} \neq u_2^{(i)}$. This violates the direct product hardness property of coset states.
- Either one of $P_{\mathbf{A}}(x_1, \mathbf{u}_1)$ or $P_{\mathbf{A}}(x_2, \mathbf{u}_2)$ is 0. In this case, we violate the constraint hiding property of constrained functionalities.

Putting both of the above cases together, we have that the advantage of the adversary in the security experiment of private constrained functionality to be negligible. We prove this formally in Lemma 40.

2.2 One-Time Programs for Constrained Functionalities

We discuss the main ideas behind proving Theorem 3. Our main approach is inspired by the construction of one-time programs for pseudorandom functions by [GLR+24]. However, in comparison with [GLR+24], not only do we generalize the class of functionalities for which we show that one-time programs exist, we also simplify their approach⁴.

Suppose we have a class of constrained functionalities $\text{CF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$. We will assume that CF satisfies *search* security. The same template can also be adapted for the case when CF satisfies *decisional* security.

To compute a one-time program for a function f_k , do the following:

⁴As a concrete example, they use a primitive referred to as *invertible* pseudorandom functions in their constructions whereas we observe that it is not required.

- As in Section 2.1, first generate n coset states, where the i^{th} coset state is $\left| A_{s_i, s'_i}^{(i)} \right\rangle = \frac{1}{\sqrt{|A_i|}} \sum_{a_i \in A_i} (-1)^{\langle a_i, s'_i \rangle} |a_i + s_i\rangle$
- Constrain the function f_k using the constraint circuit $P_{\mathbf{A}}$ to obtain $k_{\mathbf{A}}$. While the functionality of $P_{\mathbf{A}}$ is identical to its counterpart from Section 2.1, the implementation of $P_{\mathbf{A}}$ is different. To recall, the constraint $P_{\mathbf{A}}$ takes as input x along with n vectors $u^{(1)}, \dots, u^{(n)}$. It performs the following check for every i : $x^{(i)} = 0$ if and only if $u^{(i)} \in A_i + s_i$ or $x^{(i)} = 1$ if and only if $u^{(i)} \in A_i^\perp + s'_i$. If even one check is violated then it returns 0, otherwise it returns 1.

The main difference lies in the implementation of the coset membership performed inside $P_{\mathbf{A}}$. The constrained function $k_{\mathbf{A}}$ need not necessarily hide the description of the subspaces A_i ; after all CF only satisfies constrained security and need not necessarily satisfy *constrained hiding* property. To be able to make use of the unclonability property of the coset states, it is important that the adversary is not able to readily observe the description of the cosets. To address this issue, we hardwire, $\{\widehat{A_i}, \widehat{A_i}^\perp\}_i$ in $P_{\mathbf{A}}$, where $\widehat{A_i}$ is the obfuscation of a program that checks for membership in $A_i + s_i$ and moreover, $\widehat{A_i}^\perp$ is the obfuscation of a program that checks for membership in $A_i^\perp + s'_i$.

Set $\left(\left\{ \left| A_{s_i, s'_i}^{(i)} \right\rangle \right\}_i, k_{\mathbf{A}} \right)$ to be the one-time program computing f_k .

The evaluation of the one-time program is identical to its counterpart from Section 2.1.

Security. To prove security, suppose there exists a quantum polynomial time (QPT) adversary that can produce (with non-negligible advantage) two inputs $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ and two outputs y_1, y_2 such that $f_k(x_1; \mathbf{u}_1) = y_1$ and $f_k(x_2; \mathbf{u}_2) = y_2$. We first observe that if $x_1 \neq x_2$ then $\mathbf{u}_1 \neq \mathbf{u}_2$. Consider the following two cases:

- Both $P_{\mathbf{A}}(x_1, \mathbf{u}_1) = 1$ and $P_{\mathbf{A}}(x_2, \mathbf{u}_2) = 1$. Suppose $\mathbf{u}_1 = (u_1^{(1)}, \dots, u_1^{(n)})$ and $\mathbf{u}_2 = (u_2^{(1)}, \dots, u_2^{(n)})$. In this case, since $\mathbf{u}_1 \neq \mathbf{u}_2$, we have that there exists $i \in [n]$ such that $u_1^{(i)} \neq u_2^{(i)}$. This violates the following direct product hardness property of coset states [CLLZ21; CHV23]: given one copy of a coset state $\frac{1}{\sqrt{|A|}} \sum_{a \in A} (-1)^{\langle a, s' \rangle} |a + s\rangle$ and given $\widehat{A}, \widehat{A}^\perp$ (where $\widehat{A}$ is the obfuscation of the program checking membership in $A + s$, $\widehat{A}^\perp$ is the obfuscation of the program checking membership in $A^\perp + s'$), cannot produce u, v such that: (a) $u \in A + s, v \in A^\perp + s'$, (b) $u, v \in A + s$ where $u \neq v$ and, (c) $u, v \in A^\perp + s'$ where $u \neq v$.
- Either one of $P_{\mathbf{A}}(x_1, \mathbf{u}_1)$ or $P_{\mathbf{A}}(x_2, \mathbf{u}_2)$ is 0. Let's assume $P_{\mathbf{A}}(x_1, \mathbf{u}_1) = 0$. This violates the constraint hiding property of constrained functionalities since it is computationally hard to produce the output of f_k on $(x_1, \mathbf{u}_1)$ given the constrained program $k_{\mathbf{A}}$, where $P_{\mathbf{A}}(x_1, \mathbf{u}_1) = 0$.

2.3 Malicious QOTP

We consider another variant called malicious quantum one-time programs. Suppose $\mathcal{F} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$ is the class of functionalities for which we design quantum one-time programs. In a malicious quantum one-time program for $\mathcal{F}$, we guarantee security against the party generating the one-time program. Specifically, we equip the scheme with an efficient verification algorithm *Verify* that attests whether the quantum one-time program is valid or invalid. In terms

of security, we require that there is an extraction algorithm E such that conditioned on verification succeeding (i.e. valid output), E can extract an index k with probability close to 1. Intuitively, this means that if the quantum one-time program is valid then there should exist some k such that the post-verification quantum state is a valid one-time program of f_k .

We design a malicious QOTP for constrained functionalities. The starting point is the construction we sketched in Section 2.2. A natural first thought to upgrade this scheme to be secure against a malicious generator is as follows: prove using a non-interactive zero-knowledge proof (NIZK) that the quantum one-time program has been correctly computed. Unfortunately this does not work: the statement we want to prove is a quantum state and NIZKs have been mainly realized for languages and promise problems (i.e. with classical statements). Instead, we crucially make use of the structure of the quantum one-time program construction. The construction consists of two components: the first component is the set of coset states and the second component is the constrained functionality. Since the constrained functionality has a classical description, we can at least attach a NIZK to prove that this component is correctly computed. But what about the first component? here, we make use of the fact that the first component is required to be a set of coset states of the form $\{|A_{s_i, s'_i}^{(i)}\rangle\}_i$. To check whether the first component is indeed of this form, we give out obfuscated membership programs. Specifically, for every i , we provide obfuscation of membership in $A_i + s_i$ and similarly, provide obfuscation of membership in $A_i^\perp + s'_i$. It can be shown that on any state applying the following verification:

- Run the obfuscation of membership in $A_i + s_i$,
- Perform $H^{\otimes \lambda}$ (here, λ is the dimension of the vector space from which we draw A_i),
- Run the obfuscation of membership in $A_i^\perp + s'_i$,
- Perform $H^{\otimes \lambda}$.

will result in the state $|A_{s_i, s'_i}^{(i)}\rangle$ since the above verification procedure is a rank one projector $|A_{s_i, s'_i}^{(i)}\rangle\langle A_{s_i, s'_i}^{(i)}|$. Thus, these obfuscated membership programs serve the purpose of verifying whether the quantum states are of the right form. Furthermore, we will also prove using NIZK that the obfuscated membership programs themselves are correctly computed.

2.3.1 Implication to One-Message 2PC

We consider the implication of malicious one-time programs to one-message two-party secure computation (Theorem 4). The informal description of the two-party functionality $\mathcal{F}$ can be found right after Theorem 4. The construction is simple: party P_1 sends a quantum one-time program of f_k to P_2 , where k is P_1 's input. P_2 then runs the evaluation on its input x to obtain $(u, f_k(x; u))$. We note that u is a function of k, x and some internal randomness.

The simulation security against P_1 follows from the extractability property of the underlying malicious quantum one-time program. The ideal world in the simulation security definition is described as follows: P_1 sends k to the ideal (randomized) functionality, P_2 sends x and in turn P_2 receives $(u, f_k(x; u))$, where u . The operational security of the quantum one-time program implies the operational security against P_2 .

2.4 Related Work

The area of quantum one-time programs is tightly connected to unclonable cryptography, which leverages the no-cloning principle of quantum mechanics to build cryptographic primitives that are impossible to achieve classically. Some well studied primitives include public-key quantum money [Aar09; Zha21] and quantum copy-protection [Aar09; CLLZ21; AL21]. In particular, as mentioned earlier, quantum one-time programs for publicly verifiable functionalities imply public-key quantum money.

Quantum one-time programs for specific cryptographic functionalities have been studied in the past. As an example, Ben-David and Sattath [BS23] and later Coladangelo, Liu, Liu, Zhandry [CLLZ21] studied the problem of designing quantum one-time programs for signing functionalities with public verification. Behera, Sattath and Shinar [BSS21] designed quantum one-time programs for signing functionalities with private verification.

Our work attempts to put forth a characterization of the class of functionalities for which we can design quantum one-time programs in the plain model. Interestingly, a recent work by [AB24; CG24] also shows a similar characterization of the class of functionalities that can be copy-protected. Our work shows one-time programs for any constrained functionality exist whereas [AB24; CG24] shows that any puncturable functionality (a special case of constrained functionalities) can be copy-protected.

3 Preliminaries

3.1 Notation

We denote the security parameter by λ . The set $\{1, \dots, n\}$, for $n \in \mathbb{N}$, is denoted by $[n]$. Vectors are denoted by boldface font $\mathbf{A} = (A_1, \dots, A_n)$. A function $\epsilon(\lambda)$ is negligible denoted by $\text{negl}(\lambda)$ if for all polynomial $p(\lambda)$, $\epsilon(\lambda) < 1/p(\lambda)$ for sufficiently large λ . Let $x \in \{0, 1\}^n$, $x^{(1)}x^{(2)} \dots x^{(n)} = \text{Decomp}(x)$ is the binary representation of x .

3.2 Privately Constrained Pseudorandom Functions

A privately constrained pseudorandom function (PCPRF) allows an owner of a PRF key k to compute a constrained key k_P for any predicate P . The correctness property requires that given k_P , evaluator can compute PRF outputs on all inputs x such that $P(x) = 1$ (i.e. the input satisfies the predicate). The security property stipulates that the key k_P does not reveal anything about the predicate P and PRF output on an input x , such that $P(x) = 0$, remains hidden.

We recall the definition of privately constrained PRFs (PCPRF) from [BTVW17].

Definition 6. A privately constrained pseudo-random function (PCPRF) family for a class of predicates $\mathcal{P}$ is defined by algorithms (KeyGen, Eval, Constrain, CEval) where:

- **KeyGen** $(1^\lambda, 1^\ell, 1^d, 1^m)$ is a PPT algorithm that takes as input security parameter, a circuit max-length ℓ , a circuit max-depth d and an output space m , and outputs a PRF key k and public parameters pp .
- **Eval**_{pp} (k, x) is a deterministic algorithm that takes as input PRF key k and input $x \in \{0, 1\}^*$ and outputs $y \in \{0, 1\}^m$

- $\text{Constrain}_{\text{pp}}(k, P)$ is a PPT algorithm that takes as input a key k and circuit $P : \{0, 1\}^* \rightarrow \{0, 1\} \in \mathcal{P}$ and outputs constrained key k_P .
- $\text{CEval}_{\text{pp}}(k_P, x)$ is a deterministic algorithm that takes as input k_P and $x \in \{0, 1\}^*$ and outputs $y \in \{0, 1\}^m$ or $\perp$.

Additionally, it satisfies correctness: $\forall \lambda \in \mathbb{N}, x \in \{0, 1\}^n, P : \{0, 1\}^* \rightarrow \{0, 1\}$ such that $P(x) = 1$, we have

$$\Pr \left[\text{CEval}_{\text{pp}}(k_P, x) = \text{Eval}(k, x) \mid \begin{smallmatrix} (k, \text{pp}) \leftarrow \text{KeyGen}(1^\lambda) \\ k_P \leftarrow \text{Constrain}_{\text{pp}}(k, P) \end{smallmatrix} \right] = 1$$

The security game from [BTVW17] below captures the constrain privacy, security of PCPRF together. Challenger samples bits $c_1, c_2 \xleftarrow{\$} \{0, 1\}$. $\mathcal{A}$ sends predicates P_0, P_1 to a challenger $\mathcal{C}$. Bit c_1 selects which predicate is used and $\mathcal{A}$ receives k_P constrained by predicate P_{c_1} . The $\mathcal{A}$ then makes $q = \text{poly}(\lambda)$ evaluation queries $\{x_i\}_{i \in [q]}$ and receives $y_i = \text{Eval}(k, x_i)$. Finally, the adversary submits a challenge query x such that $P(x) = 0$. Bit c_1 selects whether $\mathcal{C}$ sends $y = \text{Eval}(k, x)$ or $y \xleftarrow{\$} \{0, 1\}^m$ and $\mathcal{A}$ guesses either c_1 or c_2 . An adversary making evaluation queries $\{x_i\}_{i \in [q]}$ and challenge query x is admissible if $\forall i \in [q] : x_i \neq x$ and $\forall i \in [q] : P_0(x) = P_1(x)$.

Definition 7. Consider the following game between QPT adversary $\mathcal{A}$ and a challenger $\mathcal{C}$:

1. $\mathcal{A}$ sends $1^\ell, 1^d, 1^m$ and predicates P_0, P_1 to $\mathcal{C}$.
2. $\mathcal{C}$ generates $(\text{pp}, k) \leftarrow \text{KeyGen}(1^\lambda, 1^\ell, 1^d, 1^m)$. It $c_1, c_2 \xleftarrow{\$} \{0, 1\}$. $\mathcal{C}$ generates $k_P \leftarrow \text{Constrain}_{\text{pp}}(k, P_{c_1})$ and sends (pp, k_P) to $\mathcal{A}$.
3. $\mathcal{A}$ sends $q = \text{poly}(\lambda)$ adaptive queries $\{x_i\}_{i \in [q]}$ to $\mathcal{C}$.
4. For all $i \in [q] : \mathcal{C}$ sends $y_i = \text{Eval}_{\text{pp}}(k, x_i)$ to $\mathcal{A}$.
5. $\mathcal{A}$ sends x to $\mathcal{C}$.
6. $\mathcal{C}$ sends $y \in \{0, 1\}^m$ to $\mathcal{A}$ where:

$$y = \begin{cases} \text{Eval}_{\text{pp}}(k, x) & c_2 = 0 \\ U(\{0, 1\}^m) & c_2 = 1 \end{cases}$$

where U is uniform distribution.

7. $\mathcal{A}$ outputs (j, c') where $j \in \{1, 2\}$.
8. $\mathcal{A}$ wins if $c' = c_j$.

Let $\mathcal{A}$ be an admissible adversary and $\text{Adv}_{\mathcal{A}}^{\text{PCPRF}}(1^\lambda) = |\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}|$ be the advantage of $\mathcal{A}$ in the security game. A family of PRFs is single-key privately constrained PRF (PCPRF) if for every QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{PCPRF}}(1^\lambda) = \text{negl}(\lambda)$

Theorem 8 ([BTVW17]). Assuming that learning with errors (LWE) is hard against quantum polynomial time adversaries, there exists a single-key PCPRF for P/Poly .

Remark 9. Definition 7 is stronger than the security game considered in [BTVW17]. Specifically, in the above definition, the adversary is allowed to obtain the outputs of the PRF on an input x_i wherein $P_0(x_i) = P_1(x_i) = 0$. On the other hand, in the definition of [BTVW17], on any input x_i wherein $P_0(x_i) = P_1(x_i) = 0$, the challenger returned either the PRF output or a uniformly random string. However, the proof of the construction in [BTVW17] can be adapted to satisfy Definition 7.

3.3 Indistinguishability Obfuscation

We recall the definition of post-quantum secure indistinguishability obfuscation [BGI+01; GGH+13].

Definition 10 (Indistinguishability obfuscator ($i\mathcal{O}$)). *A uniform PPT machine $i\mathcal{O}$ is called an indistinguishability obfuscator for a circuit class $\{\mathcal{C}_\lambda\}$ if the following conditions are satisfied:*

- *For all $\lambda \in \mathbb{N}$, $C \in \mathcal{C}_\lambda$, for all inputs x , we have*

$$\Pr \left[\hat{C}(x) = C(x) \mid \hat{C} \leftarrow i\mathcal{O} \left(1^\lambda, C \right) \right] = 1$$

- *For any QPT distinguisher D , there exists a negligible function α such that following hold: For all $\lambda \in \mathbb{N}$, for all pairs of circuits $C_0, C_1 \in \mathcal{C}_\lambda$, we have that if $C_0(x) = C_1(x)$ for all inputs x , then*

$$\left| \Pr[D(i\mathcal{O}(1^\lambda, C_1)) = 1] - \Pr[D(i\mathcal{O}(1^\lambda, C_0)) = 1] \right| \leq \alpha(\lambda)$$

There are several candidates of post-quantum secure $i\mathcal{O}$ for polynomial-size classical circuits [BGMZ18; AP20; DQV+21].

3.4 Subspace Hiding Obfuscator

Obfuscators for circuits that check subspace membership can be constructed from $i\mathcal{O}$ and injective one-way functions that hide the description of subspace. We recall definition of subspace hiding obfuscator from [Zha21].

Definition 11 (Subspace Hiding Obfuscator [Zha21]). *A subspace hiding obfuscator (shO) for field $\mathbb{F}$ is a PPT algorithm that takes as input description of a subspace $S \subset \mathbb{F}^\lambda$ and outputs a circuit $\hat{S}$ such that it satisfies following properties:*

- **Correctness:** *For all $x \in \mathbb{F}^\lambda$ and subspace $S \subset \mathbb{F}^\lambda$,*

$$\Pr[\hat{S}(x) = S(x) \mid \hat{S} \leftarrow \text{shO} \left(1^\lambda, S \right)] \geq 1 - \text{negl}(\lambda)$$

- **Security:** *Consider the following game between QPT adversary $\mathcal{A}$ and challenger $\mathcal{C}$:*

1. *$\mathcal{A}$ samples a subspace $S_0 \subset \mathbb{F}^\lambda$ of dimension d_0 and sends to $\mathcal{C}$.*
2. *$\mathcal{C}$ samples a uniformly random subspace $S_1 \subset \mathbb{F}^\lambda$ of dimension $d_1 \geq d_0$ such that $S_0 \subseteq S_1$.*
3. *$\mathcal{C}$ samples a random coin $b \xleftarrow{\$} \{0, 1\}$, generates $\hat{S} \leftarrow \text{shO} \left(1^\lambda, S_b \right)$ and sends $\hat{S}$ to $\mathcal{A}$.*
4. *$\mathcal{A}$ outputs $b' \in \{0, 1\}$ and wins if $b' = b$.*

Let $\text{Adv}_{\mathcal{A}}^{\text{shO}}(1^\lambda) = \left| \Pr \left[b \leftarrow \mathcal{A} \left(\hat{S}_b \right) \right] - \frac{1}{2} \right|$ be the advantage of $\mathcal{A}$ in the security game. shO is secure subspace hiding obfuscator if for all QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{shO}}(1^\lambda) = \text{negl}(\lambda)$.

Theorem 12 (Theorem 5.3 in [Zha21]). *If an injective one way function exists then any post-quantum indistinguishability obfuscator, appropriately padded, is also a secure subspace hiding obfuscator for field $\mathbb{F}$ and dimensions d_0, d_1 as long as $|\mathbb{F}|^{\lambda-d_1}$ is exponential.*

3.5 Non Interactive Zero Knowledge Proof of Knowledge

Let $R(x, w)$ be an efficiently computable relation. The language $\mathcal{L} = \{x \mid \exists w : R(x, w) = 1\}$ is in NP. We recall the definition of NIZK-PoK for language $\mathcal{L}$.

Definition 13. Let $\lambda \in \mathbb{N}$, R be a binary predicate and $\mathcal{L}$ be a language as defined above. Π is a NIZK-PoK proof system for $\mathcal{L}$ if there is a tuple of QPT algorithms (Setup, Prove, Verify) such that:

- **Setup**(1^λ) : Outputs common reference string crs .
- **Prove**(x, w, crs) : On input $x \in \mathcal{L}_{\mathcal{F}}$ and witness w , outputs proof π .
- **Verify**(x, π, crs) : Outputs $\top / \perp$

Additionally, it satisfies the following properties:

- **Completeness:** For all x, w such that $R(x, w) = 1$,

$$\Pr \left[\text{Verify}(x, \pi, \text{crs}) = \top \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ \pi \leftarrow \text{Prove}(x, w, \text{crs}) \end{array} \right] = 1$$

- **Soundness:** For all QPT $\mathcal{A}$ and $x \notin \mathcal{L}$,

$$\Pr \left[\text{Verify}(x, \pi, \text{crs}) = \perp \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

- **Knowledge Extraction:** There is a QPT extractor $E = (E_1, E_2)$ such that for all QPT $\mathcal{A}$,

$$\Pr[1 \leftarrow \mathcal{A}(\text{crs}) \mid \text{crs} \leftarrow \text{Setup}(1^\lambda)] - \Pr[1 \leftarrow \mathcal{A}(\text{crs}) \mid (\text{crs}, \tau) \leftarrow E_1(1^\lambda)] \leq \text{negl}(\lambda)$$

$$\Pr \left[\text{Verify}(x, \pi, \text{crs}) = \perp \vee R(x, w) = 1 \mid \begin{array}{l} (\text{crs}, \tau) \leftarrow E_1(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(\text{crs}) \\ w \leftarrow E_2(x, \pi, \text{crs}, \tau) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

- **Single Theorem Zero Knowledge:** For all QPT adversary $\mathcal{A}$, $\exists$ QPT $\mathcal{S}$ such that

$$\left| \Pr[1 \leftarrow \text{Expt}_{\text{Real}}^{\mathcal{A}}(1^\lambda)] - \Pr[1 \leftarrow \text{Expt}_{\text{Ideal}}^{\mathcal{A}}(1^\lambda)] \right| \leq \text{negl}(\lambda)$$

where $\text{Expt}_{\text{Real}}^{\mathcal{A}}(1^\lambda)$ and $\text{Expt}_{\text{Ideal}}^{\mathcal{A}}(1^\lambda)$ are defined as follows:

$\text{Expt}_{\text{Real}}^{\mathcal{A}}(1^\lambda)$:

1. $\text{crs} \leftarrow \text{Setup}(1^\lambda)$.
2. $(x, w, s) \leftarrow \mathcal{A}(\text{crs})$
3. $\pi \leftarrow \text{Prove}(x, w, \text{crs})$
4. Return $\mathcal{A}(p, s)$

$\text{Expt}_{\text{Ideal}}^{\mathcal{A}}(1^\lambda)$:

1. $(\text{crs}, \tau) \leftarrow \mathcal{S}(1^\lambda)$.
2. $(x, w, s) \leftarrow \mathcal{A}(\text{crs})$
3. $\pi \leftarrow \mathcal{S}(x, \text{crs}, \tau)$
4. Return $\mathcal{A}(p, s)$

3.6 Trace Distance

We assume the familiarity of readers with fundamentals of quantum computing covered in [NC11]. We recall the definition of trace distance which will be used later in defining malicious security.

Definition 14 (Trace Distance). Trace distance for two density matrices of quantum states ρ, σ is defined as

$$\text{TD}(\rho, \sigma) = \|\rho - \sigma\|_{tr} = \frac{1}{2} \sqrt{\text{Tr}[(\rho - \sigma)^\dagger (\rho - \sigma)]}$$

3.7 Direct Product Hardness of Coset States

We recall the definition and properties of coset states in this section. Consider vector space $\mathbb{F}_2^\lambda$ of dimension λ over binary field $\mathbb{F}_2$. For any subspace $A \subseteq \mathbb{F}_2^\lambda$, the orthogonal space is denoted by $A^\perp$. Moreover, $\dim(A) + \dim(A^\perp) = \lambda$ for any subspace A .

Definition 15 (Coset State). *For any subspace $A \subseteq \mathbb{F}_2^\lambda$ and $s, s' \in \mathbb{F}_2^\lambda$, the coset state is defined as*

$$|A_{s,s'}\rangle = \frac{1}{\sqrt{|A|}} \sum_{a \in A} (-1)^{\langle a, s' \rangle} |a + s\rangle$$

For any coset state $|A_{s,s'}\rangle$, applying an λ -fold Hadamard maps it to the coset state of orthogonal space $A^\perp$ i.e.

$$H^{\otimes \lambda} |A_{s,s'}\rangle = |A_{s',s}^\perp\rangle$$

Given $|A_{s,s'}\rangle$ for random subspace A of dimension $\lambda/2$ and membership oracles for cosets $A + s, A^\perp + s'$, it was shown in [CLLZ21; CHV23] that no adversary can output distinct $u, v \in \mathbb{F}_2^\lambda$ where $u, v \in A + s$ or $u, v \in A^\perp + s'$ or $u \in A + s, v \in A^\perp + s'$ except with non-negligible probability. This is called the information theoretic direct hardness property of coset states. It was further proved that computational direct product hardness property holds even if the membership oracles are replaced by obfuscated membership circuits $i\mathcal{O}(A + s), i\mathcal{O}(A^\perp + s')$ using subspace hiding obfuscator.

Theorem 16 (Direct Product Hardness [CLLZ21; CHV23]). *Let $A \subseteq \mathbb{F}_2^\lambda$ be a uniformly random subspace of dimension $\lfloor \lambda/2 \rfloor$. Given one copy of $|A_{s,s'}\rangle$ and obfuscated membership check circuits $i\mathcal{O}(A + s), i\mathcal{O}(A^\perp + s')$, no QPT $\mathcal{A}$ can output $(u, v) \in \mathbb{F}_2^\lambda \times \mathbb{F}_2^\lambda$ such that either:*

1. $u, v \in A + s$ and $u \neq v$.
2. $u, v \in A^\perp + s'$ and $u \neq v$.
3. $u \in A + s, v \in A^\perp + s'$.

except with negligible probability.

We also state a variation of direct product hardness which is proved in [CHV23].

Theorem 17 (Lemma 4.14 in [CLLZ21], Claim 11 in [CHV23]). *Let $C, A, B \subseteq \mathbb{F}_2^\lambda$ be uniformly random subspaces such that $C \subseteq A \subseteq B$ of dimensions $\lfloor \lambda/4 \rfloor, \lfloor \lambda/2 \rfloor, \lfloor 3\lambda/4 \rfloor$ respectively. Let $t = w_B + s, t' = w_{C^\perp} + s'$ for $s, s' \xleftarrow{\$} \mathbb{F}_2^\lambda$, $w_B \xleftarrow{\$} B$ and $w_{C^\perp} \xleftarrow{\$} C^\perp$. Given one copy of $|A_{s,s'}\rangle$ and B, C, t, t' , no (unbounded) $\mathcal{A}$ can output $(u, v) \in \mathbb{F}_2^\lambda \times \mathbb{F}_2^\lambda$ such that either:*

1. $u, v \in A + s$ and $u \neq v$.
2. $u, v \in A^\perp + s'$ and $u \neq v$.
3. $u \in A + s, v \in A^\perp + s'$.

except with negligible probability.

4 Definitions

4.1 Quantum One-Time Programs

Quantum one time programs for deterministic functionalities are ruled out in [BGS13]. We recall the definition of quantum one time programs for randomized functionalities from [GLR+24]. A randomized functionality is a family of functions $\mathcal{F} = \{f_k : \{0,1\}^n \times \{0,1\}^r \rightarrow \{0,1\}^m\}$ indexed by function key $k \in \{0,1\}^\lambda$. The output of function f_k on input $x \in \{0,1\}^n$ and random string $u \xleftarrow{\$} \{0,1\}^r$ is denoted by $f_k(x; u)$.

Definition 18. Let $\mathcal{F} := \{f_k : \{0,1\}^n \times \{0,1\}^r \rightarrow \{0,1\}^m\}$ be a randomized functionality indexed by $k \in \{0,1\}^\lambda$. A quantum one-time program for $\mathcal{F}$, $\mathcal{F} - \text{QOTP}$ consists of QPT algorithms (Generate, Evaluate) where:

- **Generate** $(1^\lambda, k)$ is a QPT algorithm that takes as input $k \in \{0,1\}^\lambda$ and outputs QOTP key k_{QOTP} .
- **Evaluate** (k_{QOTP}, x) is a QPT algorithm that takes as input QOTP key k_{QOTP} and $x \in \{0,1\}^n$, outputs $(y, u) \in \{0,1\}^m \times \{0,1\}^r$.

Additionally, it satisfies correctness i.e. for all $\lambda \in \mathbb{N}, k \in \{0,1\}^\lambda, x \in \{0,1\}^n$, we have

$$\Pr \left[y = f_k(x; u) \mid \begin{smallmatrix} k_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k) \\ (y, u) \leftarrow \text{Evaluate}(k_{\text{QOTP}}, x) \end{smallmatrix} \right] = 1$$

$$\left\{ \text{Evaluate}(\text{Generate}(1^\lambda, k), x) \right\} \approx_c \left\{ (f_k(x; u), u) \mid u \xleftarrow{\$} \{0,1\}^r \right\}$$

There are various security definitions for QOTP. We consider the *weak operational security* for QOTP which says that no efficient algorithm can produce outputs on two different inputs (x, u) . However, there is a weaker security definition where no efficient adversary is even able to distinguish whether $y = f_k(x; u)$ or $y \leftarrow \mathcal{D}$ (for some distribution $\mathcal{D}$) for adversarially chosen x and u . We call the former *search weak operational security* and the latter *decision weak operational security*. For certain primitives like one-time signatures, no adversary should be able to produce two messages and signatures which implies search security whereas for other primitives like pseudorandom functions, no adversary should be able to distinguish outputs from uniform distribution so it should satisfy decision security.

Definition 19 (Decision Weak Operational Security). Suppose $\mathcal{F} - \text{QOTP}$ is a quantum one-time program for $\mathcal{F} := \{f_k : \{0,1\}^n \times \{0,1\}^r \rightarrow \{0,1\}^m\}$. Let $\mathcal{D}$ be a distribution over $\{0,1\}^m$. Consider the following game between QPT adversary $\mathcal{A}$ and challenger $\mathcal{C}$:

- $\mathcal{C}$ samples $k \leftarrow \{0,1\}^\lambda$ and sends $k_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k)$ to $\mathcal{A}$.
- $\mathcal{A}$ sends (x_1, u_1) and (x_2, u_2) to $\mathcal{C}$. $\mathcal{C}$ aborts if $(x_1, u_1) = (x_2, u_2)$.
- $\mathcal{C}$ samples $b_1, b_2 \xleftarrow{\$} \{0,1\}$ and computes $y_1, y_2 \in \{0,1\}^m$ where

$$y_i = \begin{cases} f_k(x_i; u_i) & b_i = 0 \\ \mathcal{D}(\{0,1\}^m) & b_i = 1 \end{cases}$$

- $\mathcal{C}$ sends y_1, y_2 to $\mathcal{A}$.
- $\mathcal{A}$ outputs b'_1, b'_2 .
- $\mathcal{A}$ wins if $b'_1 = b_1 \wedge b'_2 = b_2$.

An adversary can win trivially by checking whether y_1 is a correct function output and guessing bit b_2 with probability $\frac{1}{2}$. Let $\text{Adv}_{\mathcal{A}}^{\mathcal{F}\text{-QOTP}}(1^\lambda) = |\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}|$ be the advantage of $\mathcal{A}$ in the security game. $\mathcal{F}\text{-QOTP}$ satisfies decision weak operational security if for any QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\mathcal{F}\text{-QOTP}}(1^\lambda) = \text{negl}(\lambda)$

Definition 20 (Search Weak Operational Security). Suppose $\mathcal{F}\text{-QOTP}$ is a quantum one-time program for $\mathcal{F} := \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$. Consider the following game between QPT adversary $\mathcal{A}$ and challenger $\mathcal{C}$:

- $\mathcal{C}$ samples $k \leftarrow \{0, 1\}^\lambda$ and sends $k_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k)$ to $\mathcal{A}$.
- $\mathcal{A}$ outputs (x_1, u_1, y_1) and (x_2, u_2, y_2) where $x_i \in \{0, 1\}^n, u_i \in \{0, 1\}^r$ and $y \in \{0, 1\}^m$.
- $\mathcal{A}$ wins if $(x_1 \neq x_2) \vee (u_1 \neq u_2)$ and $\forall i \in \{1, 2\} : y_i = f_k(x_i; u_i)$.

An adversary can win trivially by computing (x_1, u_1, y_1) from the one-time program and guessing $y_2 \in \{0, 1\}^m$ with probability $1/2^m$ (for $m = \text{poly}(\lambda)$, probability of winning is $\text{negl}(\lambda)$). Let $\text{Adv}_{\mathcal{A}}^{\mathcal{F}\text{-QOTP}}(1^\lambda) = |\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2^m}|$ be the advantage of $\mathcal{A}$ over the trivial adversary in the security game. $\mathcal{F}\text{-QOTP}$ satisfies search weak operational security if for any QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\mathcal{F}\text{-QOTP}}(1^\lambda) = \text{negl}(\lambda)$.

Remark 21. We use the same notation $\text{Adv}_{\mathcal{A}}^{\mathcal{F}\text{-QOTP}}(1^\lambda)$ for both search and decision-weak-operational security to avoid overloading the notation. The security game will be clear from the context.

4.2 Constrained Functionalities

We define constrained randomized functionalities and their properties.

Definition 22 (Constrained Functionality (CF)). Let $\text{CF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$ be a randomized functionalities indexed by $k \in \{0, 1\}^\lambda$. Let $P : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}$ be a predicate. CF is a constrained functionality if there is tuple of PPT algorithms (KeyGen, Eval, Constrain, CEval) such that:

- $\text{KeyGen}(1^\lambda)$ is a PPT algorithm that takes as input security parameter λ and outputs uniformly random $k \in \{0, 1\}^\lambda$.
- $\text{Eval}(k, (x; u))$ is a deterministic algorithm that takes as input function key $k \in \{0, 1\}^\lambda$, function input $x \in \{0, 1\}^n$, random string $u \in \{0, 1\}^r$ and outputs $y \in \{0, 1\}^m$.
- $\text{Constrain}(k, P)$ is a PPT algorithm that takes as input function key $k \in \{0, 1\}^\lambda$, predicate $P : \{0, 1\}^{n+r} \rightarrow \{0, 1\}$ and outputs constrained key k_P .
- $\text{CEval}(k_P, (x; u))$ is a deterministic algorithm that takes as input constrained key k_P , function input $x \in \{0, 1\}^n$, random string $u \in \{0, 1\}^r$ and outputs $y \in \{0, 1\}^m$.

Additionally it satisfies correctness: $\forall \lambda \in \mathbb{N}, k \in \{0, 1\}^\lambda, x \in \{0, 1\}^n, u \in \{0, 1\}^r$ and $P : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}$ such that $P(x, u) = 1$, we have

$$\Pr \left[\text{CEval}(k_P, (x; u)) = \text{Eval}(k, (x; u)) = f_k(x; u) \Big|_{k_P \leftarrow \text{Constrain}(1^\lambda, k, P)}^{k \leftarrow \text{KeyGen}(1^\lambda)} \right] = 1$$

We capture both decision and search security for constrained functionalities by a security game between challenger and adversary. The security game has two phases - (i) *evaluation phase* and (ii) *challenge phase*. The first phase is common for decision and search security games.

Evaluation Phase. The adversary receives a constrained function key k_P where P is a predicate chosen by the adversary. Then adversary makes $q = \text{poly}(\lambda)$ evaluation queries $\{(x_i, u_i)\}_{i \in [q]}$ and receives function value $\{f_k(x_i; u_i)\}_{i \in [q]}$.

Challenge Phase. In the decision security, the adversary sends a challenge query (x, u) . The challenger samples $c \xleftarrow{\$} \{0, 1\}$ and sends $f_k(x; u)$ or $y \leftarrow \mathcal{D}$ ($\mathcal{D}$ is a distribution over $\{0, 1\}^m$) to the adversary. The adversary wins if it can distinguish the two cases with non-negligible probability. In the search security, the adversary outputs (x, u, y) and wins if $y = f_k(x; u)$. Moreover, we need to restrict the queries the adversary can make otherwise it can win the security game trivially. To address this, we define an *admissible adversary* that makes permissible queries.

Definition 23 (Admissible adversary). *Let $\mathcal{A}$ be a QPT adversary that makes $\{(x_i, u_i)\}_{i \in [q]}$ evaluation queries and (x, u) (or (x, u, y)) challenge query. $\mathcal{A}$ is an admissible adversary if following conditions are satisfied:*

1. $\forall i \in [q] : (x_i, u_i) \neq (x, u)$
2. $P(x, u) = 0$.

Definition 24 (Decision CF security). *Let $\text{CF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$ be a constrained randomized functionality. Let $\mathcal{D}$ be a distribution over $\{0, 1\}^m$. Consider the following game between a QPT adversary $\mathcal{A}$ and a challenger $\mathcal{C}$:*

- $\mathcal{A}$ samples a predicate $P : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}$ and sends to challenger.
- $\mathcal{C}$ generates $k \leftarrow \text{KeyGen}(1^\lambda), k_P \leftarrow \text{Constrain}(k, P)$ and sends k_P to $\mathcal{A}$.
- $\mathcal{A}$ sends $q = \text{poly}(\lambda)$ adaptive queries $(x_i, u_i) \in \{0, 1\}^n \times \{0, 1\}^r$ to $\mathcal{C}$.
- For all $i \in [q]$: $\mathcal{C}$ sends $y_i = \text{Eval}(k, (x_i; u_i))$ to $\mathcal{A}$.
- $\mathcal{A}$ sends (x, u) to $\mathcal{C}$.
- $\mathcal{C}$ samples $c \xleftarrow{\$} \{0, 1\}$ and sends $y \in \{0, 1\}^m$ to $\mathcal{A}$ where:

$$y = \begin{cases} \text{Eval}(k, (x; u)) & c = 0 \\ \mathcal{D}(\{0, 1\}^m) & c = 1 \end{cases}$$

- $\mathcal{A}$ outputs c' .

- $\mathcal{A}$ wins if $c' = c$.

Let $\mathcal{A}$ be an admissible QPT adversary. Let $\text{Adv}_{\mathcal{A}}^{\text{CF}}(1^\lambda) = |\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}|$ be the advantage of $\mathcal{A}$ in the security game over trivial adversary that outputs random c' . CF satisfies decision CF security if for all QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{CF}}(1^\lambda) = \text{negl}(\lambda)$.

Definition 25 (Search CF security). Let $\text{CF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$ be a constrained randomized functionality. Consider the following game between a QPT adversary $\mathcal{A}$ and a challenger $\mathcal{C}$:

- $\mathcal{A}$ samples a predicate P and sends to challenger.
- $\mathcal{C}$ generates $k \leftarrow \text{KeyGen}(1^\lambda)$, $k_P \leftarrow \text{Constrain}(k, P)$ and sends k_P to $\mathcal{A}$.
- $\mathcal{A}$ sends $q = \text{poly}(\lambda)$ adaptive queries $(x_i, u_i) \in \{0, 1\}^n \times \{0, 1\}^r$ to $\mathcal{C}$.
- For all $i \in [q]$: $\mathcal{C}$ sends $y_i = \text{Eval}(k, (x_i; u_i))$ to $\mathcal{A}$.
- $\mathcal{A}$ outputs (x, u, y) .
- $\mathcal{A}$ wins if $y = f_k(x; u)$.

Let $\mathcal{A}$ be an admissible QPT adversary. Let $\nu(\lambda)$ be the winning probability of a trivial adversary that outputs (x, u, y) for a random $y \leftarrow \mathcal{D}$. Then $\text{Adv}_{\mathcal{A}}^{\text{CF}}(1^\lambda) = |\Pr[\mathcal{A} \text{ wins}] - \nu(\lambda)|$ is the advantage of $\mathcal{A}$ in the security game over the trivial adversary. CF satisfies search CF security if for all QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{CF}}(1^\lambda) = \text{negl}(\lambda)$.

Remark 26. In all of our applications, probability of winning in search security for a trivial adversary will be negligible.

4.3 Privately Constrained Functionality

We define privately constrained randomized functionalities (PCF) and their properties. PCF is a generalization of privately constrained pseudorandom functions from Section 3.2.

Definition 27 (Privately Constrained Functionality (PCF)). Let $\text{PCF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$ be a randomized functionality indexed by $k \in \{0, 1\}^\lambda$. Let $P : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}$ be a predicate. PCF is a privately constrained functionality if there is tuple of PPT algorithms $(\text{KeyGen}, \text{Eval}, \text{Constrain}, \text{CEval})$ such that:

- $\text{KeyGen}(1^\lambda)$ is a PPT algorithm that takes as input security parameter λ and outputs uniformly random $k \in \{0, 1\}^\lambda$.
- $\text{Eval}(k, (x; u))$ is a deterministic algorithm that takes as input key $k \in \{0, 1\}^\lambda$, function input $x \in \{0, 1\}^n$, random string $u \in \{0, 1\}^r$ and outputs $y \in \{0, 1\}^m$.
- $\text{Constrain}(k, P)$ is a PPT algorithm that takes as input function key k , predicate $P : \{0, 1\}^{n+r} \rightarrow \{0, 1\}$ and outputs constrained key k_P .
- $\text{CEval}(k_P, (x; u))$ is a deterministic algorithm that takes as input constrained key k_P , function input $x \in \{0, 1\}^n$, random string $u \in \{0, 1\}^r$ and outputs $y \in \{0, 1\}^m$.

Additionally, it satisfies correctness: $\forall \lambda \in \mathbb{N}, k \in \{0, 1\}^\lambda, x \in \{0, 1\}^n, u \in \{0, 1\}^r$ and $P : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}$ such that $P(x, u) = 1$, we have

$$\Pr \left[\text{CEval}(k_P, (x; u)) = \text{Eval}(k, (x; u)) = f_k(x; u) \mid \begin{smallmatrix} k \leftarrow \text{KeyGen}(1^\lambda) \\ k_P \leftarrow \text{Constrain}(1^\lambda, k, P) \end{smallmatrix} \right] = 1$$

Similar to constrained functionality, we define decision and search security with added requirement that given constrained k_P , no adversary $\mathcal{A}$ can learn about predicate P .

The security game will begin by $\mathcal{A}$ sending predicates P_0, P_1 to challenger $\mathcal{C}$. $\mathcal{C}$ will sample a random bit $c_1 \xleftarrow{\$} \{0, 1\}$, key $k \leftarrow \text{KeyGen}(1^\lambda)$ and send $k_P \leftarrow \text{Constrain}(k, P_{c_1})$ to $\mathcal{A}$. Rest of the security game will follow same as for constrained functionality where $\mathcal{A}$ queries $\{(x_i, u_i)\}_{i \in [q]}$ and learn $\{f_k(x_i; u_i)\}_{i \in [q]}$. Finally in the challenge phase, $\mathcal{C}$ samples $c_2 \xleftarrow{\$} \{0, 1\}$ and responds to the challenge query. Now the adversary has two options, it can either output $(1, c'_1)$ or $(2, c'_2)$ and $\mathcal{A}$ wins if $c'_1 = c_1$ or $c'_2 = c_2$. In the search security, $\mathcal{A}$ can either output $(1, c'_1)$ or (x, u, y) and wins if $c'_1 = c_1$ or $y = f_k(x; u)$. In addition to the restriction on admissible queries (Definition 23) for constrained functionality security, we further require that the adversary does not query on an input that satisfies only one of the predicate otherwise the adversary can learn c_0 trivially. This requirement is captured by condition (1) in definition below.

Definition 28 (Admissible adversary). *Let $\mathcal{A}$ be a QPT adversary that makes $\{(x_i, u_i)\}_{i \in [q]}$ evaluation queries and (x, u) (or (x, u, y)) challenge query. $\mathcal{A}$ is an admissible adversary if following conditions are satisfied:*

1. $\forall i \in [q] : (P_0(x_i, u_i) = 0) \Leftrightarrow (P_1(x_i, u_i) = 0)$
2. $\forall i \in [q] : (x_i, u_i) \neq (x, u)$
3. $P_0(x, u) = 0 \wedge P_1(x, u) = 0$.

Definition 29 (Decision PCF Security). *Let $\text{PCF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$ be a privately constrained randomized functionality. Let $\mathcal{D}$ be a distribution over $\{0, 1\}^m$. Consider the following game between a QPT adversary $\mathcal{A}$ and challenger $\mathcal{C}$:*

- $\mathcal{A}$ sends predicates $P_0, P_1 : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}$ to $\mathcal{C}$.
- $\mathcal{C}$ generates $k \leftarrow \text{KeyGen}(1^\lambda)$. $\mathcal{C}$ samples $c_1 \xleftarrow{\$} \{0, 1\}$ and generates $k_P \leftarrow \text{Constrain}(k, P_{c_1})$ and sends k_P to $\mathcal{A}$.
- $\mathcal{A}$ sends $q = \text{poly}(\lambda)$ adaptive queries $(x_i, u_i) \in \{0, 1\}^n \times \{0, 1\}^r$ to $\mathcal{C}$.
- For all $i \in [q]$: $\mathcal{C}$ sends $y_i = \text{Eval}(k, (x_i; u_i))$ to $\mathcal{A}$.
- $\mathcal{A}$ sends (x, u) to $\mathcal{C}$.
- $\mathcal{C}$ samples $c_2 \xleftarrow{\$} \{0, 1\}$ and sends $y \in \{0, 1\}^m$ to $\mathcal{A}$ where:

$$y = \begin{cases} \text{Eval}(k, (x; u)) & c_2 = 0 \\ \mathcal{D}(\{0, 1\}^m) & c_2 = 1 \end{cases}$$

- $\mathcal{A}$ outputs (j, c') where $j \in \{1, 2\}$.

- $\mathcal{A}$ wins if $c' = c_j$.

Let $\mathcal{A}$ be an admissible QPT adversary. Let $\text{Adv}_{\mathcal{A}}^{\text{PCF}}(1^\lambda) = |\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}|$ be the advantage of $\mathcal{A}$ in the security game over trivial adversary that outputs random c'_j . PCF satisfies decision PCF security if for all QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{PCF}}(1^\lambda) = \text{negl}(\lambda)$.

Definition 30 (Search PCF security). Let $\text{PCF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$ be a privately constrained randomized functionality. Consider the following game between a QPT adversary $\mathcal{A}$ and a challenger $\mathcal{C}$:

- $\mathcal{A}$ sends predicates $P_0, P_1 : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}$ to $\mathcal{C}$.
- $\mathcal{C}$ generates $k \leftarrow \text{KeyGen}(1^\lambda)$. $\mathcal{C}$ samples $c_1 \xleftarrow{\$} \{0, 1\}$ and generates $k_P \leftarrow \text{Constrain}(k, P_{c_1})$ and sends k_P to $\mathcal{A}$.
- $\mathcal{A}$ sends $q = \text{poly}(\lambda)$ adaptive queries $(x_i, u_i) \in \{0, 1\}^n \times \{0, 1\}^r$ to $\mathcal{C}$.
- For all $i \in [q]$: $\mathcal{C}$ sends $y_i = \text{Eval}(k, (x_i; u_i))$ to $\mathcal{A}$.
- $\mathcal{A}$ outputs either $(1, c'_1)$ or (x, u, y) .
- $\mathcal{A}$ wins if either $c'_1 = c_1$ or $y = f_k(x; u)$.

Let $\mathcal{A}$ be an admissible QPT adversary. Let $\nu(\lambda)$ be the winning probability of a trivial adversary that outputs (x, u, y) for random $y \leftarrow \mathcal{D}$. Then

$$\text{Adv}_{\mathcal{A}}^{\text{PCF}}(1^\lambda) = \left| \Pr[\mathcal{A} \text{ wins} \mid \mathcal{A} \text{ outputs } b'_1] - \frac{1}{2} \right| + |\Pr[\mathcal{A} \text{ wins} \mid \mathcal{A} \text{ outputs } (x, u, y)] - \nu(\lambda)|$$

is the advantage of $\mathcal{A}$ in the security game over trivial adversary. PCF satisfies search PCF security if for all QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{PCF}}(1^\lambda) = \text{negl}(\lambda)$.

Remark 31. In all of our applications, probability of winning for a trivial adversary that outputs (x, u, y) will be negligibly small.

5 QOTP for Privately Constrained Functionality

5.1 Construction

Let $\mathcal{F} = \{f_k : \{0, 1\}^n \times \{0, 1\}^{n\lambda} \rightarrow \{0, 1\}^m \mid k \in \{0, 1\}^\lambda\}$ be a privately constrained randomized functionality (Definition 27) that takes as input $x \in \{0, 1\}^n$ and random string $u \in \{0, 1\}^{n\lambda}$ and outputs $f_k(x; u) \in \{0, 1\}^m$. The Generate and Evaluate algorithm for QOTP of $\mathcal{F}$ will be as follows:

Generate $(1^\lambda, k)$:

1. Sample n uniformly random subspaces $A_i \subseteq \mathbb{F}_2^\lambda$ of dimension $\lfloor \lambda/2 \rfloor$. Let $\mathbf{A} = (A_1, \dots, A_n)$.
Let $\mathbf{s} = (s_1, \dots, s_n)$ and $\mathbf{s}' = (s'_1, \dots, s'_n)$ where $s_i, s'_i \xleftarrow{\$} \mathbb{F}_2^\lambda$ for all $i \in [n]$.

2. For all $i \in [n]$, generate coset states:

$$\left| A_{s,s'}^{(i)} \right\rangle = \frac{1}{2^{\lambda/4}} \sum_{a \in A_i} (-1)^{\langle a, s'_i \rangle} |a + s_i\rangle$$

3. Generate $k_{\mathbf{A}} = \mathcal{F}.\text{Constrain}(1^\lambda, k, P_{\mathbf{A}})$ where $P_{\mathbf{A}} : \{0, 1\}^n \times \{0, 1\}^{n\lambda} \rightarrow \{0, 1\}$ is the following predicate:

On input $(x, \mathbf{u})$ where $x \in \{0, 1\}^n$ and $\mathbf{u} = (u^{(1)}, \dots, u^{(n)})$, $u^{(i)} \in \mathbb{F}_2^\lambda$ do:

- Let $x^{(1)}x^{(2)} \dots x^{(n)} \leftarrow \text{Decomp}(x)$.
- For any $i \in [n]$: if $x^{(i)} = 0 \wedge u^{(i)} \notin A_i + s_i$ or $x^{(i)} = 1 \wedge u_i \notin A_i^\perp + s'_i$, output 0.
- Output 1 otherwise.

4. Output QOTP key $k_{\text{QOTP}} = \left(\left\{ \left| A_{s,s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_{\mathbf{A}} \right)$.

Evaluate(k_{QOTP}, x) :

1. Parse QOTP key k_{QOTP} as $\left(\left\{ \left| A_{s,s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_{\mathbf{A}} \right)$ and let $x^{(1)}x^{(2)} \dots x^{(n)} \leftarrow \text{Decomp}(x)$.
2. For all $i \in [n]$: Compute $|\psi_i\rangle = (H^{\otimes \lambda})^{x^{(i)}} \left| A_{s,s'}^{(i)} \right\rangle$ and measure to get $u^{(i)} \in \mathbb{F}_2^\lambda$.
3. Compute $y \leftarrow \mathcal{F}.\text{CEval}(k_{\mathbf{A}}, (x; \mathbf{u}))$ where $\mathbf{u} = (u^{(1)}, \dots, u^{(n)}) \in \{0, 1\}^{n\lambda}$.
4. Output $(y, \mathbf{u})$.

5.2 Correctness and Security

For all $\lambda \in \mathbb{N}$, $f_k \in \mathcal{F}$ and $x \in \{0, 1\}^n$, we can verify that

$$\Pr \left[y = f_k(x; \mathbf{u}) \mid \begin{smallmatrix} k_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k) \\ (y, \mathbf{u}) \leftarrow \text{Evaluate}(k_{\text{QOTP}}, x) \end{smallmatrix} \right] = 1$$

Suppose for any $i \in [n]$, $x^{(i)} = 0$. Since the coset $A_i + s_i$ is uniformly random coset and measurement gives uniformly random $u^{(i)} \in A_i + s_i$, we have

$$\left\{ \text{Evaluate}(\text{Generate}(1^\lambda, k), x) \right\} \approx_c \left\{ (f_k(x; \mathbf{u}), \mathbf{u}) \mid \mathbf{u} \xleftarrow{\$} \{0, 1\}^{n\lambda} \right\}$$

Thus, the QOTP scheme for $\mathcal{F}$ satisfies correctness(Definition 18). Next, we show that if $\mathcal{F}$ is a PCF satisfying decision PCF (search PCF resp.) security (Definition 29 and 30) then $\mathcal{F} - \text{QOTP}$ satisfies decision(search resp.) weak operational security (Definition 19 and 20).

Theorem 32. *Let $\mathcal{X} \in \{\text{decision}, \text{search}\}$. If $\mathcal{F}$ satisfies $\mathcal{X}$ -PCF security then $\mathcal{F} - \text{QOTP}$ satisfies $\mathcal{X}$ -weak operational security.*

Remark 33. *We give a proof for decision weak operational security below. The proof for search weak operational security will be exactly same except reduction to decision PCF security will be replaced by search PCF security.*

To prove Theorem 32, we need to show that the advantage of $\mathcal{A}$ in the decision weak operational security game (Definition 19) is negligible for any QPT adversary $\mathcal{A}$. To prove this result, we consider the following sequence of hybrids:

- **Hyb₀**: This is the real security game between $\mathcal{A}$ and $\mathcal{C}$. For any (unbounded) $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{Hyb}_0}(1^\lambda) = \text{Adv}_{\mathcal{A}}^{\mathcal{F}\text{-QOTP}}(1^\lambda)$.
- **Hyb₁**: This is identical to Hyb₀ except $\mathcal{C}$ additionally computes a predicate $P_{\mathbf{T}, \mathbf{V}}$ as described below. (The difference from Hyb₀ is **highlighted**):
 - $\mathcal{C}$ samples $k \leftarrow \mathcal{F}.\text{KeyGen}(1^\lambda)$ and sends $k_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k)$ to $\mathcal{A}$.
 - $\mathcal{C}$ additionally computes a predicate $P_{\mathbf{T}, \mathbf{V}} : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}$ as follows:
 - * $\mathcal{C}$ samples $2n$ uniformly random subspaces of $\mathbb{F}_2^\lambda$, $\mathbf{T} = (T_1, \dots, T_n)$, $\mathbf{V} = (V_1, \dots, V_n)$ such that $\dim(V_i) = \lfloor \lambda/4 \rfloor$, $\dim(T_i) = \lfloor 3\lambda/4 \rfloor$, and $V_i \subseteq A_i \subseteq T_i$ for all $i \in [n]$.
 - * For all $i \in [n]$: $\mathcal{C}$ samples $w_T^i \xleftarrow{\$} T_i$, $w_{V^\perp}^i \xleftarrow{\$} V_i^\perp$ and sets $t_i \leftarrow s_i + w_T^i$, $t'_i \leftarrow s'_i + w_{V^\perp}^i$.
 - * Predicate $P_{\mathbf{T}, \mathbf{V}}$ is defined as follows:
 - On input $(x, \mathbf{u}) \in \{0, 1\}^n \times \{0, 1\}^{n\lambda}$ do:
 1. Let $x^{(1)} \dots x^{(n)} \leftarrow \text{Decomp}(x)$.
 2. For all $i \in [n]$: If $x^{(i)} = 0 \wedge u^{(i)} \notin T_i + t_i$ or $x^{(i)} = 1 \wedge u^{(i)} \notin V_i^\perp + t'_i$, output 0.
 3. Output 1 otherwise.
 - $\mathcal{A}$ sends $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ to $\mathcal{C}$. $\mathcal{C}$ aborts if $(x_1, \mathbf{u}_1) = (x_2, \mathbf{u}_2)$.
 - $\mathcal{C}$ sends $y_1, y_2 \in \{0, 1\}^m$ to $\mathcal{A}$.
 - $\mathcal{A}$ outputs b'_1, b'_2 and wins if $b'_1 = b_1 \wedge b'_2 = b_2$.

Remark 34. In Hyb₁, $\mathcal{C}$ computes $P_{\mathbf{T}, \mathbf{V}}$ but the function key k is still constrained by $P_{\mathbf{A}}$.

Remark 35. Since $V_i \subseteq A_i \subseteq T_i$, $t_i \in T_i + s_i$ and $t'_i \in V_i^\perp + s'_i$, we have $A_i + s_i \subseteq B_i + t_i$ and $A_i^\perp + s'_i \subseteq V_i^\perp + t'_i$. Therefore, $P_{\mathbf{A}}(x, \mathbf{u}) = 1 \implies P_{\mathbf{T}, \mathbf{V}}(x, \mathbf{u}) = 1$ and $P_{\mathbf{T}, \mathbf{V}}(x, \mathbf{u}) = 0 \implies P_{\mathbf{A}}(x, \mathbf{u}) = 0$.

Definition 36. We define a bad event when $\mathcal{A}$ outputs at least one $(x, \mathbf{u})$ such that $P_{\mathbf{A}}(x, \mathbf{u}) = 0 \wedge P_{\mathbf{T}, \mathbf{V}}(x, \mathbf{u}) = 1$. Let $\Pr[\text{bad event} | \mathcal{A}^{\text{Hyb}}]$ be the probability that bad event event happens when $\mathcal{A}$ is in hybrid Hyb.

Lemma 37. For any (unbounded) $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{Hyb}_1}(1^\lambda) = \text{Adv}_{\mathcal{A}}^{\text{Hyb}_0}(1^\lambda)$ and $\Pr[\text{bad event} | \mathcal{A}^{\text{Hyb}_1}] = \text{negl}(\lambda)$.

Proof. The only difference between Hyb₁ and Hyb₀ is that $\mathcal{C}$ samples additional predicate $P_{\mathbf{T}, \mathbf{V}}$ internally. However, the view of $\mathcal{A}$ is identical in both hybrids and $\text{Adv}_{\mathcal{A}}^{\text{Hyb}_1}(1^\lambda) = \text{Adv}_{\mathcal{A}}^{\text{Hyb}_0}(1^\lambda)$. To bound the probability that bad event occurs, we bound the probability that $\mathcal{A}$ outputs a bad query $(x, \mathbf{u})$ such that $P_{\mathbf{A}}(x, \mathbf{u}) = 0 \wedge P_{\mathbf{T}, \mathbf{V}}(x, \mathbf{u}) = 1$ or, equivalently,

$$\exists i \in [n] : \left(x^{(i)} = 0 \wedge u^{(i)} \in (T_i + t_i) \setminus (A_i + s_i) \right) \vee \left(x^{(i)} = 1 \wedge u^{(i)} \in (V_i^\perp + t'_i) \setminus (A_i^\perp + s'_i) \right)$$

. Therefore, we have

$$\begin{aligned}
\Pr[\text{bad event} | \mathcal{A}^{\text{Hyb}_1}] &= \Pr \left[(x_1, \mathbf{u}_1) \text{ is bad} \cup (x_2, \mathbf{u}_2) \text{ is bad} \mid \mathcal{A}^{\text{Hyb}_1} \right] \\
&\leq 2 \Pr \left[(x, \mathbf{u}) \text{ is bad} \mid \mathcal{A}^{\text{Hyb}_1} \right] \\
&\leq 2 \sum_{i=1}^n \Pr \left[u^{(i)} \in (T_i + t_i) \setminus (A_i + s_i) \mid \mathcal{A}^{\text{Hyb}_1} \right] \\
&\quad + 2 \sum_{i=1}^n \Pr \left[u^{(i)} \in (V_i^\perp + t'_i) \setminus (A_i^\perp + s'_i) \mid \mathcal{A}^{\text{Hyb}_1} \right] \tag{1}
\end{aligned}$$

Since $\mathbf{T}, \mathbf{V}, \mathbf{t}, \mathbf{t}'$ are uniformly random and information theoretically hidden from $\mathcal{A}$, for all $i \in [n]$:

$$\begin{aligned}
\Pr \left[u^{(i)} \in (T_i + t_i) \setminus (A_i + s_i) \mid \mathcal{A}^{\text{Hyb}_1} \right] &\leq \Pr \left[u^{(i)} \in (T_i + t_i) \setminus (A_i + s_i) \mid A_i, s_i, s'_i \right] \\
&= \frac{2^{3\lambda/4} - 2^{\lambda/2}}{2^\lambda - 2^{\lambda/2}} \tag{2}
\end{aligned}$$

$$\begin{aligned}
\Pr \left[u^{(i)} \in (V_i^\perp + t'_i) \setminus (A_i^\perp + s'_i) \mid \mathcal{A}^{\text{Hyb}_1} \right] &\leq \Pr \left[u^{(i)} \in (V_i^\perp + t'_i) \setminus (A_i^\perp + s'_i) \mid A_i^\perp, s^i, s'_i \right] \\
&= \frac{2^{3\lambda/4} - 2^{\lambda/2}}{2^\lambda - 2^{\lambda/2}} \tag{3}
\end{aligned}$$

Hence from equation (1), (2) and (3), we have

$$\Pr \left[\text{bad event} \mid \mathcal{A}^{\text{Hyb}_1} \right] \leq 4n2^{-\lambda/4} = \text{negl}(\lambda)$$

□

- **Hyb₂**: This is same as Hyb₁ except $\mathcal{C}$ constrains the PCF key k with predicate $P_{\mathbf{T}, \mathbf{V}}$ as described below. (The difference from Hyb₀ is **highlighted**):

- $\mathcal{C}$ samples $k \leftarrow \mathcal{F}.\text{KeyGen}(1^\lambda)$. When executing $\text{Generate}(1^\lambda, k)$, $\mathcal{C}$ samples $\mathbf{A}, \mathbf{T}, \mathbf{V}$ and $P_{\mathbf{T}, \mathbf{V}}$ as before but **computes $k_{\mathbf{T}, \mathbf{V}} \leftarrow \mathcal{F}.\text{Constrain}(1^\lambda, k, P_{\mathbf{T}, \mathbf{V}})$** .
- **$\mathcal{C}$ sets $k_{\text{QOTF}} = \left(\left\{ \left| A_{s, s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_{\mathbf{T}, \mathbf{V}} \right)$ and sends k_{QOTF} to $\mathcal{A}$.**
- $\mathcal{A}$ sends $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ to $\mathcal{C}$. $\mathcal{C}$ aborts if $(x_1, \mathbf{u}_1) = (x_2, \mathbf{u}_2)$.
- $\mathcal{C}$ samples $b_1, b_2 \xleftarrow{\$} \{0, 1\}$ and sends y_1, y_2 to $\mathcal{A}$.
- $\mathcal{A}$ outputs b'_1, b'_2 and wins if $b'_1 = b_1 \wedge b'_2 = b_2$.

Lemma 38. *For any QPT $\mathcal{A}$, $\Pr[\text{bad event} \mid \mathcal{A}^{\text{Hyb}_2}] = \text{negl}(\lambda)$*

Proof. From lemma 37, we know that $\Pr[\text{bad event} \mid \mathcal{A}^{\text{Hyb}_1}] = \nu(\lambda)$ for negligible $\nu(\cdot)$. Assume that $\Pr[\text{bad event} \mid \mathcal{A}^{\text{Hyb}_2}] = 1/p(\lambda)$ for polynomial $p(\cdot)$. Then we can construct an adversary $\mathcal{B}$ that can win in the decision PCF security game (Definition 29) for $\mathcal{F}$ with non-negligible advantage as follows:

Description of $\mathcal{B}$:

1. $\mathcal{B}$ samples vectors of subspaces $\mathbf{A}, \mathbf{T}, \mathbf{V}$ and sends $P_{\mathbf{A}}, P_{\mathbf{T}, \mathbf{V}}$ to challenger $\mathcal{C}$.
2. $\mathcal{C}$ samples $k \leftarrow \mathcal{F}.\text{KeyGen}(1^\lambda), c_1 \xleftarrow{\$} \{0, 1\}$ and sends $k_P \leftarrow \mathcal{F}.\text{Constrain}(k, P)$ to $\mathcal{B}$. $P = P_{\mathbf{A}}$ if $c_1 = 0$ and $P = P_{\mathbf{T}, \mathbf{V}}$ otherwise.
3. $\mathcal{B}$ computes $k_{\text{QOTP}} = \left(\left\{ \left| A_{s, s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_P \right)$ and sends to $\mathcal{A}$.
4. $\mathcal{B}$ receives $(x_1, \mathbf{u}_1)$ and $(x_2, \mathbf{u}_2)$ from $\mathcal{A}$.
5. If bad event occurs, output $(1, c'_1) = (1, 1)$.
6. Output $(1, c'_1) = (1, 1)$ otherwise.

If $c_1 = 0$, $\mathcal{A}$ is in Hyb_1 otherwise $\mathcal{A}$ is in Hyb_2 . Therefore, we have

$$\begin{aligned}
\text{Adv}_{\mathcal{B}}^{\text{PCF-QOTP}}(1^\lambda) &= |\Pr[c'_1 = 1 | c_1 = 1] - \Pr[c'_1 = 1 | c_1 = 0]| \\
&= \left| \Pr[\text{bad event} \mid \mathcal{A}^{\text{Hyb}_2}] - \Pr[\text{bad event} \mid \mathcal{A}^{\text{Hyb}_1}] \right| \\
&= \left| \frac{1}{p(\lambda)} - \nu(\lambda) \right| \\
&= 1/\text{poly}(\lambda)
\end{aligned}$$

Hence $\mathcal{B}$ wins decision PCF security game with non-negligible advantage. Therefore, $\Pr[\text{bad event} \mid \mathcal{A}^{\text{Hyb}_2}] = \text{negl}(\lambda)$. $\square$

Lemma 39. For any QPT $\mathcal{A}$, $\left| \text{Adv}_{\mathcal{A}}^{\text{Hyb}_2}(1^\lambda) - \text{Adv}_{\mathcal{A}}^{\text{Hyb}_1}(1^\lambda) \right| = \text{negl}(\lambda)$.

Proof. Suppose there is an adversary $\mathcal{A}$ that can distinguish Hyb_1 and Hyb_2 with advantage $1/p(\lambda)$ for polynomial $p(\cdot)$. Then we construct an adversary that wins in the decision PCF security (Definition 29) as follows:

Description of $\mathcal{B}$:

1. $\mathcal{B}$ samples $\mathbf{A}, \mathbf{T}, \mathbf{V}$ and sends $P_{\mathbf{A}}, P_{\mathbf{T}, \mathbf{V}}$ to challenger $\mathcal{C}$.
2. $\mathcal{C}$ samples $k \leftarrow \mathcal{F}.\text{KeyGen}(1^\lambda), c_1 \xleftarrow{\$} \{0, 1\}$ and sends $k_P \leftarrow \mathcal{F}.\text{Constrain}(k, P)$ to $\mathcal{B}$. $P = P_{\mathbf{A}}$ if $c_1 = 0$ and $P = P_{\mathbf{T}, \mathbf{V}}$ otherwise.
3. $\mathcal{B}$ computes $k_{\text{QOTP}} = \left(\left\{ \left| A_{s, s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_P \right)$ and sends to $\mathcal{A}$.
4. $\mathcal{B}$ receives $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ from $\mathcal{A}$ and does the following:
 - (a) If bad event happens, $\mathcal{B}$ outputs $(0, b'_1 \xleftarrow{\$} \{0, 1\})$.
 - (b) Otherwise, $\mathcal{B}$ sets evaluation query set to be $\{(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)\}$ and sends to $\mathcal{C}$.
5. $\mathcal{C}$ sends $y_1, y_2 \in \{0, 1\}^m$ to $\mathcal{B}$ where $y_i = \mathcal{F}.\text{Eval}(k, (x_i; \mathbf{u}_i))$.

6. $\mathcal{B}$ samples $b_1, b_2 \xleftarrow{\$} \{0, 1\}$ and sends z_1, z_2 to $\mathcal{A}$ where

$$z_j = \begin{cases} y_j & b_j = 0 \\ \mathcal{D}(\{0, 1\}^m) & b_j = 1 \end{cases}$$

7. $\mathcal{A}$ sends c'_1 to $\mathcal{B}$.

8. $\mathcal{B}$ outputs $(1, c'_1)$ and wins if $c'_1 = c_1$.

We do a case-by-case analysis:

- **Case 1:** $\exists j \in \{1, 2\} : P_{\mathbf{A}}(x_j, \mathbf{u}_j) = 0 \wedge P_{\mathbf{T}, \mathbf{V}}(x_j, \mathbf{u}_j) = 1$. This is the **bad event** and the probability that this happens is $\text{negl}(\lambda)$ in both hybrids. Let $\Pr[\text{Case 1} | \text{Hyb}_1] \leq \nu(\lambda)$ and $\Pr[\text{Case 1} | \text{Hyb}_2] \leq \nu(\lambda)$ for negligible $\nu(\cdot)$.
- **Case 2:** $\forall j \in \{1, 2\} : P_{\mathbf{A}}(x_j, \mathbf{u}_j) = 1 \vee P_{\mathbf{T}, \mathbf{V}}(x_j, \mathbf{u}_j) = 0$. Since $P_{\mathbf{A}}(x_j, \mathbf{u}_j) = 1 \implies P_{\mathbf{T}, \mathbf{V}}(x_j, \mathbf{u}_j) = 1$ and $P_{\mathbf{T}, \mathbf{V}}(x_j, \mathbf{u}_j) = 0 \implies P_{\mathbf{A}}(x_j, \mathbf{u}_j) = 0$ (See Remark 35), the set of evaluation queries $\{(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)\}$ is admissible (Definition 28). $\mathcal{C}$ sends $y_j = \mathcal{F}.\text{Eval}(k, (x_j; \mathbf{u}_j))$ to $\mathcal{B}$. $\mathcal{B}$ samples $b_1, b_2 \xleftarrow{\$} \{0, 1\}$ and sends z_j to $\mathcal{A}$ distributed independent of the c_1 sampled by $\mathcal{C}$. Therefore, if $c_1 = 0$, $\mathcal{A}$ is in Hyb_1 otherwise $\mathcal{A}$ is in Hyb_2 and $\mathcal{B}$ guesses correct c_1 with probability $1/2 + 1/p(\lambda)$ (the distinguishing advantage of $\mathcal{A}$ is $1/p(\lambda)$).

Now, we can compute winning probability and advantage of $\mathcal{B}$:

$$\begin{aligned} \Pr[\mathcal{B} \text{ wins}] &= \Pr[\text{Case 1}] \cdot \Pr[\mathcal{B} \text{ wins in Case 1}] + \Pr[\text{Case 2}] \cdot \Pr[\mathcal{B} \text{ wins in Case 2}] \\ &\geq (1 - \nu(\lambda)) \cdot \left(\frac{1}{2} + \frac{1}{p(\lambda)} \right) \\ &\geq \frac{1}{2} + \frac{1}{p'(\lambda)} \end{aligned}$$

for some polynomial $p'(\lambda)$ and $\text{Adv}_{\mathcal{B}}^{\text{PCF}}(1^\lambda) \geq 1/p'(\lambda)$ which is a contradiction. Therefore, Hyb_1 and Hyb_2 are computationally indistinguishable and for all QPT $\mathcal{A}$,

$$\left| \text{Adv}_{\mathcal{A}}^{\text{Hyb}_2}(1^\lambda) - \text{Adv}_{\mathcal{A}}^{\text{Hyb}_1}(1^\lambda) \right| = \text{negl}(\lambda)$$

□

Finally, we show that no efficient adversary can win in Hyb_2 with non-negligible advantage.

Lemma 40. *For any QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{Hyb}_2}(1^\lambda) = \text{negl}(\lambda)$*

Proof. Suppose for contradiction, there is a QPT $\mathcal{A}$ that wins in Hyb_2 with probability $1/2 + 1/p(\lambda)$ for polynomial $p(\cdot)$. Then, we can construct an adversary $\mathcal{B}$ that wins in the decision PCF security (Definition 29) of $\mathcal{F}$ as follows:

Description of $\mathcal{B}$:

1. $\mathcal{B}$ samples $\mathbf{A}, \mathbf{T}, \mathbf{V}$ and sends $P_{\mathbf{T}, \mathbf{V}}, P_{\mathbf{T}, \mathbf{V}}$ to challenger $\mathcal{C}$.

2. $\mathcal{C}$ samples $k \leftarrow \text{KeyGen}(1^\lambda)$, $c_1 \xleftarrow{\$} \{0, 1\}$ and sends $k_P \leftarrow \text{Constrain}(k, P_{\mathbf{T}, \mathbf{V}})$. Since $\mathcal{B}$ sent same predicate, k_P is independent of c_1 .
3. $\mathcal{B}$ sends $k_{\text{QOTF}} = \left(\left\{ \left| A_{s, s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_P \right)$ to $\mathcal{A}$.
4. $\mathcal{B}$ receives $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ from $\mathcal{A}$ and does the following:
 - (a) If $(x_1, \mathbf{u}_1) = (x_2, \mathbf{u}_2)$, $\mathcal{B}$ aborts.
 - (b) If $\forall j \in [2] : P_{\mathbf{T}, \mathbf{V}}(x_j, \mathbf{u}_j) = 1$, $\mathcal{B}$ outputs $(1, c'_1 \xleftarrow{\$} \{0, 1\})$.
 - (c) If $\exists j \in [2] : P_{\mathbf{T}, \mathbf{V}}(x_j, \mathbf{u}_j) = 0$, $\mathcal{B}$ sets the evaluation query set to $\{(x_{2-j+1}, \mathbf{u}_{2-j+1})\}$ and the challenge query is $(x_j, \mathbf{u}_j)$.
5. $\mathcal{B}$ sends the evaluation query set $\{(x_{2-j+1}, \mathbf{u}_{2-j+1})\}$ to $\mathcal{C}$.
6. $\mathcal{C}$ sends $y_{2-j+1} = \mathcal{F}.\text{Eval}(k, (x_{2-j+1}, \mathbf{u}_{2-j+1}))$ to $\mathcal{B}$.
7. $\mathcal{B}$ sends challenge query $(x_j, \mathbf{u}_j)$ to $\mathcal{C}$.
8. $\mathcal{C}$ samples $c_2 \xleftarrow{\$} \{0, 1\}$ and sends y_j where

$$y_j = \begin{cases} \mathcal{F}.\text{Eval}(k, (x_j, \mathbf{u}_j)) & c_2 = 0 \\ \mathcal{D}(\{0, 1\}^m) & c_2 = 1 \end{cases}$$

9. $\mathcal{B}$ samples $b_{2-j+1} \xleftarrow{\$} \{0, 1\}$ and sends z_1, z_2 to $\mathcal{A}$ where $z_j = y_j$ and

$$z_{2-j+1} = \begin{cases} y_{2-j+1} & b_{2-j+1} = 0 \\ \mathcal{D}(\{0, 1\}^m) & b_{2-j+1} = 1 \end{cases}$$

10. $\mathcal{A}$ sends b'_1, b'_2 to $\mathcal{B}$.
11. $\mathcal{B}$ outputs $(2, b'_j)$

We do a case-by-case analysis:

- **Case 1:** $(x_1, \mathbf{u}_1) = (x_2, \mathbf{u}_2)$. Since probability $\mathcal{A}$ wins is $\frac{1}{2} + \frac{1}{p(\lambda)}$, $\Pr[\text{Case 1}] \leq \frac{1}{2} - \frac{1}{p(\lambda)}$.
- **Case 2:** $\forall j \in \{1, 2\} : P_{\mathbf{T}, \mathbf{V}}(x_j, \mathbf{u}_j) = 1$. Suppose $P_{\mathbf{A}}(x_1, \mathbf{u}_1) = 0$. Then $(x_1, \mathbf{u}_1)$ is a *bad* query and, by Lemma 38, the probability that $\mathcal{A}$ outputs such a query is negligible. Therefore $\forall j \in \{1, 2\} : P_{\mathbf{A}}(x_j, \mathbf{u}_j) = 1$ except with negligible probability. Since $(x_1, \mathbf{u}_1) \neq (x_2, \mathbf{u}_2)$, there is a $i \in [n]$ such that one of the following holds:

1. $u_1^{(i)}, u_2^{(i)} \in A_i + s_i$ and $u_1^{(i)} \neq u_2^{(i)}$
2. $u_1^{(i)}, u_2^{(i)} \in A_i^\perp + s'_i$ and $u_1^{(i)} \neq u_2^{(i)}$
3. For some $j \in \{1, 2\}$, $u_j^{(i)} \in A_i + s_i$ and $u_{2-j+1}^{(i)} \in A_i^\perp + s'_i$

However, by direct product hardness property of coset states (Theorem 17), probability $\mathcal{A}$ outputs such $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ given $\mathbf{T}, \mathbf{V}, \mathbf{t}, \mathbf{t}'$ in clear in is also negligibly small. Therefore, $\Pr[\text{Case 2}] = \nu(\lambda)$ for negligible $\nu(\cdot)$.

- **Case 3:** $\exists j \in \{1, 2\} : P_{\mathbf{T}, \mathbf{V}}(x_j, \mathbf{u}_j) = 0$. WLOG assume that $P_{\mathbf{T}, \mathbf{V}}(x_2, \mathbf{u}_2) = 0$ and $P_{\mathbf{T}, \mathbf{V}}(x_1, \mathbf{u}_1)$ can be either 0 or 1. The evaluation query set will be $\{(x_1, \mathbf{u}_1)\}$, the challenge query will be $(x_2, \mathbf{u}_2)$ and $\mathcal{B}$ is an admissible adversary (Definition 28) for PCF security game. $\mathcal{C}$ sends $y_1 = \mathcal{F}.\text{Eval}(k, (x_1; \mathbf{u}_1))$ for the evaluation query. To answer the challenge query, $\mathcal{C}$ samples $c_2 \xleftarrow{\$} \{0, 1\}$ and sends $y_2 = \mathcal{F}.\text{Eval}(k, (x_2; \mathbf{u}_2))$ if $c_2 = 0$ and $y_2 \leftarrow \mathcal{D}(\{0, 1\}^m)$ otherwise. So, $z_1 = \mathcal{F}.\text{Eval}(k, (x_1; \mathbf{u}_1))$ if $b_1 = 0$ and $z_2 \leftarrow \mathcal{D}$ otherwise. Similarly, $z_2 = \mathcal{F}.\text{Eval}(k, (x_2; \mathbf{u}_2))$ if $c_2 = 0$ and $z_2 \leftarrow \mathcal{D}$ otherwise. This is same as $\mathcal{B}$ sampling b_1, c_2 as a challenger in the weak operational security game. Therefore, $b'_1 = b_1 \wedge b'_2 = c_2$ with probability $\frac{1}{2} + \frac{1}{p'(\lambda)}$ for some polynomial $p'(\cdot)$ and we have

$$\Pr[b'_2 = c_2 \mid \text{Case 3}] \geq \Pr[b'_1 = b_1 \wedge b'_2 = c_2 \mid \text{Case 3}] = \frac{1}{2} + \frac{1}{p'(\lambda)}$$

$$\Pr[\text{Case 3}] = 1 - \Pr[\text{Case 1}] - \Pr[\text{Case 2}] \geq \frac{1}{2} + \frac{1}{q(\lambda)}$$

for some polynomial $p'(\cdot), q(\cdot)$.

Computing the winning probability of $\mathcal{B}$:

$$\begin{aligned} \Pr[\mathcal{B} \text{ wins}] &= \sum_{i=1}^3 \Pr[\mathcal{B} \text{ wins} \mid \text{Case } i] \cdot \Pr[\text{Case } i] \\ &\geq \left(\frac{1}{2} + \frac{1}{q(\lambda)} \right) \cdot \left(\frac{1}{2} + \frac{1}{p'(\lambda)} \right) \\ &= \frac{1}{2} + \frac{1}{\text{poly}(\lambda)} \end{aligned}$$

and $\text{Adv}_{\mathcal{B}}^{\text{PCF}}(1^\lambda) = 1/\text{poly}(\lambda)$. Thus, $\mathcal{B}$ wins the decision PCF security game with non-negligible advantage which is a contradiction. Therefore, $\text{Adv}_{\mathcal{A}}^{\text{Hyb}_2}(1^\lambda) = \text{negl}(\lambda)$ for all QPT $\mathcal{A}$. $\square$

Proof of Theorem 32. From Lemma 37, 39 and 37, we have $\text{Adv}_{\mathcal{A}}^{\mathcal{F}-\text{QOTP}}(1^\lambda) = \text{negl}(\lambda)$ and $\mathcal{F} - \text{QOTP}$ satisfies decision weak operational security. $\square$

5.3 QOTP for Pseudorandom Functions

Theorem 41 ([BTVW17]). *Assuming that learning with errors is hard against quantum polynomial time algorithms, there is a privately constrained pseudorandom function PCPRF for predicates with circuit size polynomial in input length that satisfies decision PCF security (Definition 29).*

As a corollary of Theorem 41 and 32, we obtain a quantum one-time program for pseudorandom functions from quantum hardness of LWE.

6 QOTP for Constrained Functionalities

6.1 Construction

Let $\mathcal{F} = \{f_k : \{0,1\}^n \times \{0,1\}^{n\lambda} \rightarrow \{0,1\}^m\}$ be a constrained randomized functionality (Definition 22). Let $i\mathcal{O}$ be a quantum secure indistinguishability obfuscator (Definition 10) and shO be a subspace hiding obfuscator (Definition 11) for $\mathbb{F}_2^\lambda$. For a subspace $A \subseteq \mathbb{F}_2^\lambda$, $s \in \mathbb{F}_2^\lambda$ let $i\mathcal{O}(A + s)$ be the obfuscated membership check circuit for the coset $A + s$ as defined in Theorem 16. The Generate and Evaluate algorithm for quantum one-time program for $\mathcal{F}$ will be as follows.

Generate($1^\lambda, k$):

1. Sample n uniformly random subspaces $S_i \subseteq \mathbb{F}_2^\lambda$ of dimension $\lfloor \lambda/2 \rfloor$. Let $\mathbf{A} = (A_1, \dots, A_n)$. Let $\mathbf{s} = (s_1, \dots, s_n)$ and $\mathbf{s}' = (s'_1, \dots, s'_n)$ where $s_i, s'_i \xleftarrow{\$} \mathbb{F}_2^\lambda$ for all $i \in [n]$.
2. For all $i \in [n]$, generate coset states:

$$\left| A_{s,s'}^{(i)} \right\rangle = \frac{1}{2^{\lambda/4}} \sum_{a \in A_i} (-1)^{\langle a, s'_i \rangle} |a + s_i\rangle$$

3. Generate the obfuscated coset membership circuits

$$\forall i : \hat{A}_i = i\mathcal{O}(A_i + s_i) \quad \text{and} \quad \hat{A}_i^\perp = i\mathcal{O}(A_i^\perp + s'_i)$$

Let $\hat{\mathbf{A}} = (\hat{A}_1, \dots, \hat{A}_n)$ and $\hat{\mathbf{A}}^\perp = (\hat{A}_1^\perp, \dots, \hat{A}_n^\perp)$.

4. Generate $k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} = \text{CF.Constrain}(k, P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp})$ where $P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp} : \{0,1\}^n \times \{0,1\}^{n\lambda} \rightarrow \{0,1\}$ is the following predicate:
On input $x \in \{0,1\}^n$ and $\mathbf{u} = (u^{(1)}, \dots, u^{(n)})$, $u^{(i)} \in \mathbb{F}_2^\lambda$ do:
 - Let $x^1 x^2 \dots x^n \leftarrow \text{Decomp}(x)$.
 - If for any $i \in [n]$, $(x^{(i)} = 0) \wedge (\hat{A}_i(u^{(i)}) = 0)$ or $(x^{(i)} = 1) \wedge (\hat{A}_i^\perp(u^{(i)}) = 0)$, output 0.
 - Output 1 otherwise.

5. Output QOTP key $k_{\text{QOTP}} = \left(\left\{ \left| A_{s,s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \right)$.

Evaluate(k_{QOTP}, x):

1. Parse QOTP key k_{QOTP} as $\left(\left\{ \left| A_{s,s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \right)$ and let $x^1 x^2 \dots x^n \leftarrow \text{Decomp}(x)$.
2. For all $i \in [n]$: Compute $|\psi_i\rangle = (H^{\otimes \lambda})^{x^i} \left| A_{s,s'}^{(i)} \right\rangle$ and measure to get $u^{(i)} \in \mathbb{F}_2^\lambda$.
3. Compute $y \leftarrow \mathcal{F}.\text{CEval}(k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}}, (x; \mathbf{u}))$ where $\mathbf{u} = (u^{(1)}, \dots, u^{(n)})$.
4. Output $(y, \mathbf{u})$.

6.2 Correctness and Security

Similar to the correctness proof of QOTP for privately constrained functionality, one can show that

$$\Pr \left[y = f_k(x; \mathbf{u}) \mid \begin{smallmatrix} k_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k) \\ (y, \mathbf{u}) \leftarrow \text{Evaluate}(k_{\text{QOTP}}, x) \end{smallmatrix} \right] = 1$$

$$\left\{ \text{Evaluate}(\text{Generate}(1^\lambda, k), x) \right\} \approx_c \left\{ (f_k(x; \mathbf{u}), \mathbf{u}) \mid \mathbf{u} \xleftarrow{\$} \{0, 1\}^{n\lambda} \right\}$$

Next, we show that if $\mathcal{F}$ is a constrained randomized functionality satisfying decision CF (search CF resp.) security then $\mathcal{F} - \text{QOTP}$ satisfies decision (search resp.) weak operational security.

Theorem 42. *Let $\mathcal{X} \in \{\text{decision}, \text{search}\}$. If $\mathcal{F}$ satisfies $\mathcal{X}$ -CF security, $i\mathcal{O}$ is a quantum secure indistinguishability obfuscator and shO is secure subspace hiding obfuscator, then $\mathcal{F} - \text{QOTP}$ satisfies $\mathcal{X}$ -weak operational security.*

Remark 43. *We give the proof for decision weak operational security of $\mathcal{F} - \text{QOTP}$ below. The proof for search weak operational security will be same except the reduction to decision CF security will be replaced by reduction to search CF security.*

Proof. Suppose there is a QPT $\mathcal{A}$ that can win in decision weak operational security game (Definition 24) with non negligible advantage i.e. $\text{Adv}_{\mathcal{A}}^{\mathcal{F}-\text{QOTP}}(1^\lambda) = 1/p(\lambda)$ for polynomial $p(\cdot)$. Then we construct an adversary $\mathcal{B}$ that wins the CF security (definition 24) with non-negligible advantage.

Description of $\mathcal{B}$:

1. $\mathcal{B}$ samples subspaces $\mathbf{A} = (A_1, \dots, A_n)$ and vectors $\mathbf{s} = (s_1, \dots, s_n), \mathbf{s}' = (s'_1, \dots, s'_n)$. $\mathcal{B}$ generates coset states $\left\{ \left| A_{s, s'}^{(i)} \right\rangle \right\}_{i \in [n]}$.
2. $\mathcal{B}$ generates obfuscated membership check circuit for coset states $\hat{\mathbf{A}} = (\hat{A}_1, \dots, \hat{A}_n)$ and $\hat{\mathbf{A}}^\perp = (\hat{A}_1^\perp, \dots, \hat{A}_n^\perp)$ where
$$\forall i : \hat{A}_i = i\mathcal{O}(A_i + s_i) \quad \text{and} \quad \hat{A}_i^\perp = i\mathcal{O}(A_i^\perp + s'_i)$$
3. $\mathcal{B}$ sends predicate $P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}$ to challenger $\mathcal{C}$.
4. $\mathcal{C}$ generates $k \leftarrow \mathcal{F}.\text{KeyGen}(1^\lambda)$ and sends the constrained key $k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \leftarrow \mathcal{F}.\text{Constrain}(k, P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp})$ to $\mathcal{B}$.
5. $\mathcal{B}$ sends $k_{\text{QOTP}} = \left(\left\{ \left| A_{s, s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \right)$ to $\mathcal{A}$.
6. $\mathcal{B}$ receives $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ from $\mathcal{A}$ and does the following:
 - (a) If $(x_1, \mathbf{u}_1) = (x_2, \mathbf{u}_2)$, $\mathcal{B}$ aborts.
 - (b) If $\forall j \in [2] : P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}(x_j, \mathbf{u}_j) = 1$, $\mathcal{B}$ aborts.
 - (c) If $\exists j \in [2] : P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}(x_j, \mathbf{u}_j) = 0$, $\mathcal{B}$ sets the evaluation query set to be $\{(x_{2-j+1}, \mathbf{u}_{2-j+1})\}$ and the challenge query is $(x_j, \mathbf{u}_j)$.

7. $\mathcal{B}$ sends the evaluation query set $\{(x_{2-j+1}, \mathbf{u}_{2-j+1})\}$ to $\mathcal{C}$.
8. $\mathcal{C}$ sends $y_{2-j+1} = \mathcal{F}.\text{Eval}(k, (x_{2-j+1}; \mathbf{u}_{2-j+1}))$ to $\mathcal{B}$.
9. $\mathcal{B}$ sends challenge query $(x_j, \mathbf{u}_j)$ to $\mathcal{C}$.
10. $\mathcal{C}$ samples $c \xleftarrow{\$} \{0, 1\}$ and sends y_j where

$$y_j = \begin{cases} \mathcal{F}.\text{Eval}(k, (x_j; \mathbf{u}_j)) & c = 0 \\ \mathcal{D}(\{0, 1\}^m) & c = 1 \end{cases}$$

11. $\mathcal{B}$ samples $b_{2-j+1} \xleftarrow{\$} \{0, 1\}$ and sends z_1, z_2 to $\mathcal{A}$ where $z_j = y_j$ and

$$z_{2-j+1} = \begin{cases} y_{2-j+1} & b_{2-j+1} = 0 \\ \mathcal{D}(\{0, 1\}^m) & b_{2-j+1} = 1 \end{cases}$$

12. $\mathcal{A}$ sends b'_1, b'_2 to $\mathcal{B}$.
13. $\mathcal{B}$ outputs b'_j .

We do a case by case analysis:

- **Case 1:** $(x_1, \mathbf{u}_1) = (x_2, \mathbf{u}_2)$. Since $\mathcal{A}$ is successful with probability $\frac{1}{2} + \frac{1}{p(\lambda)}$, $\Pr[\text{Case 1}] \leq \frac{1}{2} - \frac{1}{p(\lambda)}$.
- **Case 2:** $\forall j \in \{1, 2\} : P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}(x_j, \mathbf{u}_j) = 1$. As $(x_1, \mathbf{u}_1) \neq (x_2, \mathbf{u}_2)$, there is a $i \in [n]$ such that one of the following holds:
 1. $u_1^{(i)}, u_2^{(i)} \in A_i + s_i$ and $u_1^{(i)} \neq u_2^{(i)}$
 2. $u_1^{(i)}, u_2^{(i)} \in A_i^\perp + s'_i$ and $u_1^{(i)} \neq u_2^{(i)}$
 3. For some $j \in \{1, 2\}$, $u_j^{(i)} \in A_i + s_i$ and $u_{2-j+1}^{(i)} \in A_i^\perp + s'_i$

By the direct hardness property of coset states (Theorem 16), the probability that $\mathcal{A}$ outputs such $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ that satisfy any of the above condition given obfuscated membership circuits $\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp$ is negligibly small. Therefore, $\Pr[\text{Case 2}] = \nu(\lambda)$ for negligible $\nu(\cdot)$.

- **Case 3:** $\exists j \in \{1, 2\} : P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}(x_j, \mathbf{u}_j) = 0$. WLOG assume that $P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}(x_2, \mathbf{u}_2) = 0$ and $P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}(x_1, \mathbf{u}_1)$ can be either 0 or 1. The evaluation query set will be $\{(x_1, \mathbf{u}_1)\}$, the challenge query will be $(x_2, \mathbf{u}_2)$ and $\mathcal{B}$ is an admissible adversary (Definition 23) for CF security game. $\mathcal{C}$ sends $y_1 = \mathcal{F}.\text{Eval}(k, (x_1; \mathbf{u}_1))$ for the evaluation query. To answer the challenge query, $\mathcal{C}$ samples $c \xleftarrow{\$} \{0, 1\}$ and sends $y_2 = \mathcal{F}.\text{Eval}(k, (x_2; \mathbf{u}_2))$ if $c = 0$ and $y_2 \leftarrow \mathcal{D}(\{0, 1\}^m)$ otherwise. So, $z_1 = \mathcal{F}.\text{Eval}(k, (x_1; \mathbf{u}_1))$ if $b_1 = 0$ and $z_2 \leftarrow \mathcal{D}$ otherwise. Similarly, $z_2 = \mathcal{F}.\text{Eval}(k, (x_2; \mathbf{u}_2))$ if $c = 0$ and $z_2 \leftarrow \mathcal{D}$ otherwise. This is same as $\mathcal{B}$ sampling b_1, c as a challenger in the weak operational security game. Therefore, $b'_1 = b_1 \wedge b'_2 = c$ with probability $\frac{1}{2} + \frac{1}{p'(\lambda)}$ for some polynomial $p'(\cdot)$ and we have

$$\Pr[b'_2 = c \mid \text{Case 3}] \geq \Pr[b'_1 = b_1 \wedge b'_2 = c \mid \text{Case 3}] = \frac{1}{2} + \frac{1}{p'(\lambda)}$$

$$\Pr[\text{Case 3}] = 1 - \Pr[\text{Case 1}] - \Pr[\text{Case 2}] \geq \frac{1}{2} + \frac{1}{q(\lambda)}$$

for some polynomial $p'(\cdot), q(\cdot)$.

Computing the winning probability of $\mathcal{B}$:

$$\begin{aligned} \Pr[\mathcal{B} \text{ wins}] &= \sum_{i=1}^3 \Pr[\mathcal{B} \text{ wins} | \text{Case } i] \cdot \Pr[\text{Case } i] \\ &\geq \left(\frac{1}{2} + \frac{1}{q(\lambda)} \right) \cdot \left(\frac{1}{2} + \frac{1}{p'(\lambda)} \right) \\ &= \frac{1}{2} + \frac{1}{\text{poly}(\lambda)} \end{aligned}$$

and $\text{Adv}_{\mathcal{B}}^{\text{CF}}(1^\lambda) = 1/\text{poly}(\lambda)$. Thus $\mathcal{B}$ wins the decision CF security game with non-negligible advantage which is a contradiction. Therefore, $\text{Adv}_{\mathcal{A}}^{\mathcal{F}\text{-QOTP}}(1^\lambda) = \text{negl}(\lambda)$ for all QPT $\mathcal{A}$ and $\mathcal{F}$ -QOTP scheme satisfies decision weak operational security. $\square$

7 Malicious Security

One natural extension of quantum one-time programs that we consider is security against malicious program generators. While the QOTP generator in existing definitions is trusted, in some applications such as secure multi-party computation however, there is no guarantee that the generator behaves honestly. Concretely, the generator of a QOTP for a class of functions $\mathcal{F}$ could generate a one-time program for a function $f \notin \mathcal{F}$. We begin by defining valid QOTP keys for a functionality $\mathcal{F}$ which in turn will be helpful in defining malicious secure QOTP.

Let $\mathcal{F} = \{f_k\}$ be a keyed randomized functionality where membership in functionality $f_k \in \mathcal{F}$ can be checked efficiently given function key k . We define a malicious secure quantum one-time program in such a way that the evaluator can verify the quantum one-time program and if the verification passes, then the guarantee is that the QOTP corresponds to some $f_k \in \mathcal{F}$.

Definition 44. Let $\mathcal{F} = \{f_k\}$ be a keyed randomized functionality. Let $\mathcal{K}_k$ be the set of all quantum one-time programs for $f_k \in \mathcal{F}$ i.e.

$$\mathcal{K}_k := \{\rho \mid \rho \leftarrow \mathcal{F}\text{-QOTP.Generate}(1^\lambda, k)\}$$

The set of all possible QOTP keys for $\mathcal{F}$ will be $\mathcal{K} = \cup_{f_k \in \mathcal{F}} \mathcal{K}_k$. A QOTP key output by a malicious generator k_{QOTP} is corrupted if $k_{\text{QOTP}} \notin \mathcal{K}$ or alternatively $\text{TD}(k_{\text{QOTP}}, \mathcal{K}) = \min_{\rho \in \mathcal{K}} \text{TD}(k_{\text{QOTP}}, \rho) \neq 0$ where TD is the trace distance.

Definition 45 (Malicious Secure Quantum One-Time Program). Let $\mathcal{F} := \{f_k : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^m\}$ be a family of randomized functions indexed by $k \in \{0, 1\}^\lambda$ which is equipped with an efficient membership checking algorithm. A malicious secure quantum one-time program for $\mathcal{F}$, $\mathcal{F}_{\text{mal}}\text{-QOTP}$ is a tuple of algorithm (Setup, Generate, Evaluate, Verify) where:

- **Setup**(1^λ) is a PPT algorithm that takes as input security parameter λ and outputs a common reference string crs.

- $\text{Generate}(1^\lambda, k, \text{crs})$ is a QPT algorithm that takes as input security parameter λ , common reference string crs and outputs k_{QOTP} and π_{QOTP} .
- $\text{Evaluate}(k_{\text{QOTP}}, x)$ is a QPT algorithm that takes as input $k_{\text{QOTP}}, x \in \{0, 1\}^n$ and outputs $(y \in \{0, 1\}^m, u \in \{0, 1\}^{n\lambda})$.
- $\text{Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs})$ is a QPT algorithm that takes as input $k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}$ and outputs $\top/\perp$ and ρ_{QOTP} . Since Verify performs measurement on k_{QOTP} , ρ_{QOTP} is the residual state post verification.

Additionally, it satisfies the following properties:

- **Completeness:** For all $\lambda \in \mathbb{N}$, $\text{crs} \leftarrow \text{Setup}(1^\lambda)$, $f_k \in \mathcal{F}$ and $(k_{\text{QOTP}}, \pi_{\text{QOTP}}) \leftarrow \text{Generate}(1^\lambda, k, \text{crs})$, we have

$$\Pr[\text{Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}) = (\top, \rho_{\text{QOTP}}) \wedge \text{TD}(\rho_{\text{QOTP}}, k_{\text{QOTP}}) = 0] \geq 1 - \text{negl}(\lambda)$$

- **Correctness:** If $\text{TD}(\rho_{\text{QOTP}}, \mathcal{K}) = 0$, then $\exists f_k \in \mathcal{F}$ such that $\forall x \in \{0, 1\}^n$

$$\Pr[y = f_k(x; u) \mid (y, u) \leftarrow \text{Evaluate}(\rho_{\text{QOTP}}, x)] \geq 1 - \text{negl}(\lambda)$$

- **Key Extraction:** There is a QPT extractor $E = (E_1, E_2)$ such that for all QPT $\mathcal{A}$ such that

$$\left| \Pr[1 \leftarrow \mathcal{A}(\text{crs}) \mid \text{crs} \leftarrow \text{Setup}(1^\lambda)] - \Pr[1 \leftarrow \mathcal{A}(\text{crs}) \mid (\text{crs}, \tau) \leftarrow E_1(1^\lambda)] \right| = \text{negl}(\lambda)$$

$$\Pr \left[b = \perp \vee \rho_{\text{QOTP}} \in \mathcal{K}_k \mid \begin{array}{l} (\text{crs}, \tau) \leftarrow E_1(1^\lambda) \\ (k_{\text{QOTP}}, \pi_{\text{QOTP}}) \leftarrow \mathcal{A}(\text{crs}) \\ (b, \rho_{\text{QOTP}}) \leftarrow \text{Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}) \\ k \leftarrow E_2(\rho_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}, \tau) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

That is either Verify outputs $\perp$ or E_2 outputs k where $\rho_{\text{QOTP}} \in \mathcal{K}_k$ and $f_k \in \mathcal{F}$.

- **Decision(Search)-Weak-Operational Security:** For all QPT $\mathcal{A}$, advantage of $\mathcal{A}$ in winning the weak operational security game (Definition 19, 20) is negligible: $\text{Adv}_{\mathcal{A}}^{\mathcal{F}_{\text{mal}} - \text{QOTP}}(1^\lambda) = \text{negl}(\lambda)$.

To upgrade our QOTP scheme for constrained functionality from Section 6.1 to malicious security, the generator will additionally provide a proof that the $k_{\text{QOTP}} = \left(\left\{ \left| A_{s, s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \right)$ is honestly generated. We can break the verification into two parts. Firstly, the evaluator should be able to verify that the quantum states $\left\{ \left| A_{s, s'}^{(i)} \right\rangle \right\}$ are indeed some coset states without knowing $\mathbf{A}, \mathbf{s}, \mathbf{s}'$ in clear. Secondly, the evaluator should be able to verify that that $k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}}$ is constrained using the same cosets $\{A_i + s_i, A_i^\perp + s'_i\}_{i \in [n]}$.

The main observation is that if generator gives obfuscated coset membership testing programs $\{i\mathcal{O}(A_i + s_i), i\mathcal{O}(A_i^\perp + s'_i)\}_{i \in [n]}$ with k_{QOTP} , the evaluator can verify that the states $\left\{ \left| A_{s, s'}^{(i)} \right\rangle \right\}$ and obfuscated circuits correspond to same cosets. Moreover, the evaluator can learn constrain $P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}$ in clear and this does not affect the reduction of weak operational security to CF security of $\mathcal{F}$ as the adversary is allowed to know the constraining predicate in clear in CF security game (Definition 24). Finally, to ensure that $k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}}$ is generated from some k such that $f_k \in \mathcal{F}$ and predicate $P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}$, generator is required to submit a NIZK proof which can be verified by the evaluator. We prove that these two checks are enough to achieve security against a malicious generator.

Verification of Coset States. Suppose $\mathbf{A} = (A_1, \dots, A_n)$ are subspaces $A_i \subseteq \mathbb{F}_2^\lambda$ and obfuscated circuits $\widehat{\mathbf{A}}, \widehat{\mathbf{A}}^\perp$ correspond to coset states

$$\forall i \in [n] : s_i, s'_i \in \mathbb{F}_2^\lambda, \quad \widehat{A}_i \leftarrow i\mathcal{O}(A_i + s_i), \quad \widehat{A}_i^\perp \leftarrow i\mathcal{O}(A_i^\perp + s'_i)$$

Let $\{|\psi_i\rangle\}_{i \in [n]}$ are some arbitrary quantum states supported on $\mathbb{F}_2^\lambda$. Given $(\widehat{\mathbf{A}}, \widehat{\mathbf{A}}^\perp)$, we want to verify that the state $|\psi_i\rangle$ is the coset state $|A_{s,s'}^{(i)}\rangle$. This can be done simply using the technique to verify quantum money states [AC12] as follows:

VerifyState:

On input $|\psi\rangle$, $A_0 \leftarrow i\mathcal{O}(A + s)$, $A_1 \leftarrow i\mathcal{O}(A^\perp + s')$ do,

1. Apply unitary $U : |x\rangle |0\rangle \mapsto |x\rangle |A_0(x)\rangle$ to $|\psi\rangle |0\rangle$ and measure last qubit. Let $|\psi'\rangle$ be the remaining state. If output of measurement is 0, output $(\perp, |\psi'\rangle)$.
2. Apply unitary $(H^{\otimes \lambda} \otimes I)V(H^{\otimes \lambda} \otimes I)$ to $|\psi'\rangle |0\rangle$ where $V : |x\rangle |0\rangle \mapsto |x\rangle |A_1(x)\rangle$. Measure the last qubit. Let $|\psi^v\rangle$ be the remaining state.
3. If output of measurement is 0, output $(\perp, |\psi^v\rangle)$.
4. Output $(\top, |\psi^v\rangle)$ otherwise.

Lemma 46. $\Pr[\text{VerifyState accepts } |\psi_i\rangle] = \left| \langle \psi_i | A_{s,s'}^{(i)} \rangle \right|^2$. If VerifyState outputs $(\top, |\psi^v\rangle)$ then $|\psi^v\rangle = |A_{s,s'}^{(i)}\rangle$.

Let $P_{A,s} = \sum_{a \in A} |a + s\rangle \langle a + s|$ and $P_{A^\perp, s'} = \sum_{a \in A^\perp} |a + s'\rangle \langle a + s'|$. If VerifyState outputs $\top$, $|\psi^v\rangle = H^{\otimes \lambda} P_{A^\perp, s'} H^{\otimes \lambda} P_{A,s} |\psi\rangle$. $H^{\otimes \lambda} P_{A^\perp, s'} H^{\otimes \lambda} P_{A,s}$ is a rank 1 projector $|A_{s,s'}\rangle \langle A_{s,s'}|$. Therefore, $|\psi^v\rangle = |A_{s,s'}\rangle$ irrespective of initial state $|\psi\rangle$. Thus, if VerifyState outputs $\top$ on all states $|\psi_i\rangle$, it is guaranteed that all the states are coset states $\left\{ |A_{s,s'}^{(i)}\rangle \right\}_{i \in [n]}$.

Verification of obfuscated circuits and constrained key. The generator has to give a proof that the obfuscated coset membership circuits $(\widehat{\mathbf{A}}, \widehat{\mathbf{A}}^\perp)$ correspond to coset states and constrained key $k_{P_{\widehat{\mathbf{A}}, \widehat{\mathbf{A}}^\perp}}$ is correctly generated for some $f_k \in \mathcal{F}$. Suppose the $k_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k)$ uses algorithm **CosetSamp** which takes randomness r_1 and outputs the description of $2n$ random cosets $\{A_i + s_i, A_i^\perp + s'_i\}_{i \in [n]}$ where $\dim(A_i) = \lfloor \lambda/2 \rfloor$. Then $x = \left(\left\{ \widehat{T}_i, \widehat{V}_i \right\}_i, k_C \right)$ is correctly generated if $\exists w : R(x, w) = 1$ where $R(x, w)$ is the following efficiently computable relation:

On input x, w do

1. Parse $x = \left(\left\{ \widehat{T}_i, \widehat{V}_i^\perp \right\}_i, k_C \right)$ and $w = r_1 || r_2 || r_3 || r_4 || k$.
2. Parse $r_2 = r_{2,1} || \dots || r_{2,n}$ and $r_3 = r_{3,1} || \dots || r_{3,n}$.
3. If $f_k \notin \mathcal{F}$, output 0 (Membership check).
4. Generate $\mathbf{A}, \mathbf{A}^\perp, \mathbf{s}, \mathbf{s}' \leftarrow \text{CosetSamp}(r_1)$.

5. For all $i \in [n]$, Generate $\hat{A}_i \leftarrow i\mathcal{O}(A_i + s_i; r_{2,i})$ and $\hat{A}_i^\perp \leftarrow i\mathcal{O}(A_i^\perp + s'_i; r_{3,i})$.
6. For any $i \in [n]$: if $\hat{T}_i \neq \hat{A}_i$ or $\hat{V}_i \neq \hat{A}_i^\perp$, output 0.
7. Compute the predicate $P_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}}$.
8. Compute $k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \leftarrow \text{Constrain}(k, P_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}}; r_4)$. If $k_C \neq k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}}$, output 0.
9. Otherwise output 1.

Now, we can define a language $\mathcal{L}_{\mathcal{F}}$ such that $\mathcal{L}_{\mathcal{F}} := \{x \mid \exists w : R(x, w) = 1\}$. $\mathcal{L}_{\mathcal{F}}$ is in NP since given a witness w , membership of x in $\mathcal{L}_{\mathcal{F}}$ can be verified in polynomial time by computing $R(x, w)$. Let Π be a NIZK-PoK proof system (Definition 13) for $\mathcal{L}_{\mathcal{F}}$. To prove that the obfuscated coset membership circuit and constrained key $(\{\hat{T}_i, \hat{V}_i\}_i, k_C)$ are correctly generated, generator submits a proof of correctness which can be efficiently checked by the evaluator.

7.1 Construction

Let $\mathcal{F} = \{f_k : \{0, 1\}^n \times \{0, 1\}^{n\lambda} \rightarrow \{0, 1\}^m \mid k \in \{0, 1\}^\lambda\}$ be a constrained functionality (Definition 22) and $\mathcal{F} - \text{QOTP}$ be the quantum one-time program from section 6.1. Let Π be NIZK-PoK proof system (Definition 13) for $\mathcal{L}_{\mathcal{F}}$. We describe malicious secure QOTP $\mathcal{F}_{\text{mal}} - \text{QOTP}$ below.

Setup(1^λ):

1. Generate $\text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda)$.
2. Output crs

Generate($1^\lambda, k, \text{crs}$):

1. Sample $r = r_1 || r_2 || r_3 || r_4$ where $r_2 = r_{2,1} || \dots || r_{2,n}$ and $r_3 = r_{3,1} || \dots || r_{3,n}$.
2. Compute $k_{\text{QOTP}} \leftarrow \mathcal{F} - \text{QOTP}.\text{Generate}(1^\lambda, k; r)$ as follows:

$$\begin{aligned}
\mathbf{A}, \mathbf{A}^\perp, \mathbf{s}, \mathbf{s}' &= \text{CosetSamp}(r_1) \\
\hat{\mathbf{T}} &= (i\mathcal{O}(A_1 + s_i; r_{2,1}), \dots, i\mathcal{O}(A_n + s_n; r_{2,n})) \\
\hat{\mathbf{A}}^\perp &= (i\mathcal{O}(A_1^\perp + s'_i; r_{3,1}), \dots, i\mathcal{O}(A_n^\perp + s'_n; r_{3,n})) \\
k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} &\leftarrow \text{Constrain}(1^\lambda, k, P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}, r_4) \\
k_{\text{QOTP}} &= \left(\left\{ |A_{s,s'}^{(i)}\rangle \right\}_{i \in [n]}, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \right)
\end{aligned}$$

3. Set $w \leftarrow r_1 || \dots || r_4 || k$.
4. Compute $\pi \leftarrow \Pi.\text{Prove}(\{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}}\}, w, \text{crs})$. Let $\pi_{\text{QOTP}} = (\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp, \pi)$.
5. Output $k_{\text{QOTP}}, \pi_{\text{QOTP}}$.

Evaluate(k_{QOTP}, x):

1. Compute $(y, \mathbf{u}) \leftarrow \mathcal{F} - \text{QOTP.Evalute}(k_{\text{QOTP}}, x)$.
2. Output $(y, \mathbf{u})$.

Verify($k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}$):

1. Parse $k_{\text{QOTP}} = (|\psi_i\rangle_i, k_C)$, $\pi_{\text{QOTP}} = (\hat{\mathbf{T}}, \hat{\mathbf{V}}, \pi)$ and $\hat{\mathbf{T}} = (\hat{T}_1, \dots, \hat{T}_n)$, $\hat{\mathbf{V}} = (\hat{V}_1, \dots, \hat{V}_n)$.
2. Let $x \leftarrow (\hat{\mathbf{T}}, \hat{\mathbf{V}}, k_C)$
3. Compute $b \leftarrow \Pi.\text{Verify}(x, \pi, \text{crs})$.
4. If $b = \perp$, output $\perp$. Otherwise continue.
5. For all $i \in [n]$ do:
 - (a) Run $(b, |\psi_i^v\rangle) \leftarrow \text{VerifyState}(|\psi_i\rangle, \hat{T}_i, \hat{V}_i)$.
 - (b) If $b = \perp$, output $\perp$. Otherwise continue.
6. Output $(\top, |\psi_i^v\rangle_{i \in [n]})$.

7.2 Correctness and Security

Theorem 47. *Let $\mathcal{X} \in \{\text{decision}, \text{search}\}$. If $\mathcal{F}$ satisfies $\mathcal{X}$ -CF security (Definition 24), $i\mathcal{O}$ is a indistinguishability obfuscator and Π is a NIZK-PoK then $\mathcal{F}_{\text{mal}} - \text{QOTP}$ is malicious secure quantum one-time program for $\mathcal{F}$ in CRS model assuming the existence of quantum secure indistinguishability obfuscator and injective one-way functions.*

Remark 48. *Since both $\mathcal{F}_{\text{mal}} - \text{QOTP}$ and $\mathcal{F} - \text{QOTP}$ have Generate, Evaluate algorithms, we use $\mathcal{F} - \text{QOTP}.\text{Algorithm}$ to refer to algorithm of $\mathcal{F} - \text{QOTP}$ and we use Algorithm to refer to algorithm of $\mathcal{F}_{\text{mal}} - \text{QOTP}$.*

Proof. Completeness. If $k_{\text{QOTP}}, \pi_{\text{QOTP}}$ are honestly generated, then $k_{\text{QOTP}} = \left(\left\{ |A_{s,s'}^{(i)}\rangle \right\}_{i \in [n]}, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \right)$ and $\pi_{\text{QOTP}} = (\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp, \pi)$. By completeness of NIZK and Lemma 46, $\text{Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}) = \top$ with probability 1 and $\rho_{\text{QOTP}} = k_{\text{QOTP}}$. Therefore,

$$\Pr[\text{Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}) = (\top, \rho_{\text{QOTP}}) \wedge \text{TD}(\rho_{\text{QOTP}}, k_{\text{QOTP}}) = 0] \geq 1 - \text{negl}(\lambda)$$

Correctness. If $\text{TD}(\rho_{\text{QOTP}}, \mathcal{K}) = 0, \exists f_k \in \mathcal{F} : \rho_{\text{QOTP}} \in \mathcal{K}_k$. Therefore, there is a random string r such that $\rho_{\text{QOTP}} \leftarrow \mathcal{F} - \text{QOTP.Generate}(1^\lambda, k; r)$ and by correctness of $\mathcal{F} - \text{QOTP}$ for constrained functionalities (Theorem 42), we have $\forall x \in \{0, 1\}^n$:

$$\Pr[y = f_k(x; \mathbf{u}) \mid (y, \mathbf{u}) \leftarrow \text{Evaluate}(\rho_{\text{QOTP}}, x)] \geq 1 - \text{negl}(\lambda)$$

Key Extraction: Let $k_{\text{QOTP}} = (|\psi_i\rangle_i, k_C)$, $\pi_{\text{QOTP}} = (\hat{\mathbf{T}}, \hat{\mathbf{V}}, \pi)$. Suppose verification passes i.e. $\text{Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}) = (\top, \rho_{\text{QOTP}})$. Then $\Pi.\text{Verify}(x, \pi, \text{crs}) = \top$ where $x = (\hat{\mathbf{T}}, \hat{\mathbf{V}}, k_C)$ and

by the knowledge extraction property of Π , there are efficient E'_1, E'_2 such that $(\text{crs}, \tau) \leftarrow E'_1(1^\lambda)$ such that

$$\left| \Pr \left[1 \leftarrow \mathcal{A}(\text{crs}) \mid \text{crs} \leftarrow \Pi.\text{Setup}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathcal{A}(\text{crs}) \mid (\text{crs}, \tau) \leftarrow E'_1(1^\lambda) \right] \right| = \text{negl}(\lambda)$$

$$\Pr \left[R(x, w) = 1 \mid w \leftarrow E'_2(x, \pi, \text{crs}, \tau) \right] \geq 1 - \text{negl}(\lambda)$$

where $w = (r_1 || \dots || r_4 || k)$. The key extractor for malicious QOTP will be $E = (E_1, E_2)$ described below.

Description of $E_1(1^\lambda)$:

- Output $(\text{crs}, \tau) \leftarrow E'_1(1^\lambda)$.

Description of $E_2(\rho_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}, \tau)$:

- Parse $\rho_{\text{QOTP}} = \left(|\psi_i\rangle_{i \in [n]}, k_C \right)$ and $\pi_{\text{QOTP}} = (\hat{\mathbf{T}}, \hat{\mathbf{V}}, \pi)$.
- Let $x = (\hat{\mathbf{T}}, \hat{\mathbf{V}}, k_C)$. Compute $w \leftarrow E'_2(x, \pi, \text{crs}, \tau)$.
- Parse $w = r_1 || r_2 || r_3 || r_4 || k$.
- Output k .

Conditioned on the fact that $\text{Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}) = \top$, $\rho_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k; r_1 || \dots || r_4)$ where $(r_1 || \dots || r_4 || k) \leftarrow E'_2(x, \pi, \text{crs}, \tau)$ and we have

$$\left| \Pr \left[1 \leftarrow \mathcal{A}(\text{crs}) \mid \text{crs} \leftarrow \text{Setup}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathcal{A}(\text{crs}) \mid (\text{crs}, \tau) \leftarrow E_1(1^\lambda) \right] \right| = \text{negl}(\lambda)$$

$$\Pr \left[b = \perp \vee \rho_{\text{QOTP}} \in \mathcal{K}_k \mid \begin{array}{c} (\text{crs}, \tau) \leftarrow E_1(1^\lambda) \\ (k_{\text{QOTP}}, \pi_{\text{QOTP}}) \leftarrow \mathcal{A}(\text{crs}) \\ (b, \rho_{\text{QOTP}}) \leftarrow \text{Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}) \\ k \leftarrow E_2(\rho_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}, \tau) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

Decision/Search weak operational security. We prove that $\mathcal{F}_{\text{mal}} - \text{QOTP}$ satisfies decision weak operational security (Definition 19) in the CRS model. The proof for search weak operational security (Definition 20) will be identical except the reduction to decision CF security (Definition 24) will be replaced by reduction to search CF security (Definition 25) of $\mathcal{F}$. The security game for $\mathcal{F}_{\text{mal}} - \text{QOTP}$ will be identical to $\mathcal{F} - \text{QOTP}$ except the challenger will receive $(k_{\text{QOTP}}, \pi_{\text{QOTP}})$ instead of just k_{QOTP} . On a high level, since Π is zero-knowledge, giving π_{QOTP} to $\mathcal{A}$ does not give any additional advantage to $\mathcal{A}$ and the security still holds.

Consider the following sequence of hybrids:

- **Hyb₀:** This is the real experiment between $\mathcal{A}$ and challenger $\mathcal{C}$:
 - $\mathcal{C}$ runs $\text{crs} \leftarrow \text{Setup}(1^\lambda)$ and sends crs to $\mathcal{A}$.
 - $\mathcal{C}$ generates $k \leftarrow \mathcal{F}.\text{KeyGen}(1^\lambda)$ and sends $(k_{\text{QOTP}}, \pi_{\text{QOTP}}) \leftarrow \text{Generate}(1^\lambda)$ to $\mathcal{A}$.

- $\mathcal{A}$ sends $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ to $\mathcal{C}$. $\mathcal{C}$ aborts if $(x_1, \mathbf{u}_1) = (x_2, \mathbf{u}_2)$.
- $\mathcal{C}$ samples $b_1, b_2 \xleftarrow{\$} \{0, 1\}$ and computes $y_1, y_2 \in \{0, 1\}^m$ where

$$y_i = \begin{cases} f_k(x_i; \mathbf{u}_i) & b_i = 0 \\ \mathcal{D}(\{0, 1\}^m) & b_i = 1 \end{cases}$$

- $\mathcal{A}$ outputs b'_1, b'_2 and wins if $b'_1 = b_1 \wedge b'_2 = b_2$.

Since Hyb_0 is exactly same as decision weak operational security game (Definition 19), we have

Lemma 49. $\text{Adv}_{\mathcal{A}}^{\text{Hyb}_0}(1^\lambda) = \text{Adv}_{\mathcal{A}}^{\mathcal{F}\text{-mal-QOTP}}(1^\lambda)$

- **Hyb₁:** This is same as Hyb_0 except π_{QOTP} is generated by simulator $\mathcal{S}$ as described below. (The difference from Hyb_0 are **highlighted**):

- $\mathcal{C}$ runs $(\text{crs}, \tau) \leftarrow \mathcal{S}(1^\lambda)$ and sends crs to $\mathcal{A}$.
- $\mathcal{C}$ generates $k \leftarrow \mathcal{F}.\text{KeyGen}(1^\lambda)$.
- $\mathcal{C}$ generates $(k_{\text{QOTP}}, \pi'_{\text{QOTP}}) \leftarrow \text{Generate}'(1^\lambda, k)$ where k_{QOTP} is generated same as before and π'_{QOTP} is generated using the NIZK proof simulator $\mathcal{S}$ as follows:

1. Let $k_{\text{QOTP}} = \left(\left\{ \left| A_{s,s'}^{(i)} \right\rangle \right\}_{i \in [n]}, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \right)$.
2. Let $x = \left(\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \right)$.
3. $\pi'_{\text{QOTP}} \leftarrow \mathcal{S}(x, \text{crs}, \tau)$.

and sends $(k_{\text{QOTP}}, \pi'_{\text{QOTP}})$ to $\mathcal{A}$.

- $\mathcal{A}$ sends $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ to $\mathcal{C}$. $\mathcal{C}$ aborts if $(x_1, \mathbf{u}_1) = (x_2, \mathbf{u}_2)$.
- $\mathcal{C}$ samples $b_1, b_2 \xleftarrow{\$} \{0, 1\}$ and sends y_1, y_2 to $\mathcal{A}$.
- $\mathcal{A}$ outputs b'_1, b'_2 and wins if $b'_1 = b_1 \wedge b'_2 = b_2$.

Lemma 50. For any QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{Hyb}_1}(1^\lambda) = \text{Adv}_{\mathcal{A}}^{\text{Hyb}_0}(1^\lambda) + \text{negl}(\lambda)$

This lemma follows from the fact that $\text{Hyb}_0 \approx_c \text{Hyb}_1$. To prove by contradiction, suppose there is an adversary $\mathcal{A}$ that can distinguish Hyb_0 and Hyb_1 with non-negligible probability. Then, we can construct a QPT $\mathcal{B}$ that can distinguish $\text{Expt}^{\text{Real}}$ and $\text{Expt}^{\text{Ideal}}$ defined in Definition 13 as follows:

Description of $\mathcal{B}$:

1. Challenger $\mathcal{C}$ samples $b \xleftarrow{\$} \{0, 1\}$ and generates $\text{crs} \leftarrow \text{Setup}(1^\lambda)$ if $b = 0$, and $(\text{crs}, \tau) \leftarrow \mathcal{S}(1^\lambda)$ if $b = 1$.
2. $\mathcal{C}$ sends crs to $\mathcal{B}$.
3. $\mathcal{B}$ generates $k \leftarrow \mathcal{F}.\text{KeyGen}(1^\lambda)$ and $(k_{\text{QOTP}}, \hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp) \leftarrow \mathcal{F} - \text{QOTP.Generate}(1^\lambda, k)$. Let w be the witness (randomness used to generate k_{QOTP}).
4. $\mathcal{B}$ sends (x, w) to $\mathcal{A}$ where $x = \left(\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \right)$.

5. $\mathcal{C}$ sends π to $\mathcal{B}$ where

$$\pi \leftarrow \begin{cases} \text{Prove}(x, w, \text{crs}) & b = 0 \\ \mathcal{S}(x, \text{crs}, \tau) & b = 1 \end{cases}$$

6. $\mathcal{B}$ sends $k_{\text{QOTP}}, \pi_{\text{QOTP}}$ to $\mathcal{A}$.

7. $\mathcal{A}$ sends $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ to $\mathcal{B}$.

8. $\mathcal{B}$ samples $b_1, b_2 \xleftarrow{\$} \{0, 1\}$ and sends y_1, y_2 to $\mathcal{A}$.

9. $\mathcal{A}$ sends b' to $\mathcal{B}$.

10. $\mathcal{B}$ outputs b' and wins if $b' = b$.

$\mathcal{A}$ outputs $b' = 0(b' = 1)$ if it is in $\text{Hyb}_0(\text{Hyb}_1)$ with non-negligible probability. If $b = 0$, $\pi \leftarrow \Pi.\text{Prove}(x, w, \text{crs})$ and $\mathcal{A}$ is in Hyb_0 . If $b = 1$, $\pi \leftarrow \mathcal{S}(x, \text{crs}, \tau)$ and $\mathcal{A}$ is exactly in Hyb_1 . Thus, $\mathcal{B}$ can distinguish $\text{Expt}^{\text{Real}}$ and $\text{Expt}^{\text{Ideal}}$ with non-negligible probability which is a contradiction.

Lemma 51. For all QPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{Hyb}_1}(1^\lambda) = \text{negl}(\lambda)$

Assume for contradiction that there is an adversary $\mathcal{A}$ such that $\text{Adv}_{\mathcal{A}}^{\text{Hyb}_1}(1^\lambda) = 1/p(\lambda)$ for polynomial $p(\cdot)$. Then we can construct a QPT $\mathcal{B}$ such that it violates the decision CF security (Definition 24) of $\mathcal{F}$ i.e $\text{Adv}_{\mathcal{A}}^{\text{CF}}(1^\lambda) = 1/p(\lambda)$ as follows:

Description of $\mathcal{B}$:

1. $\mathcal{B}$ generates $(\text{crs}, \tau) \leftarrow \mathcal{S}(1^\lambda)$.
2. $\mathcal{B}$ generates obfuscated coset membership circuits $\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp$ and sends predicate $P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}$ to challenger $\mathcal{C}$.
3. $\mathcal{B}$ receives constrained key $k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}}$ from $\mathcal{C}$.
4. Let $x \leftarrow (\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}})$. $\mathcal{B}$ computes

$$\begin{aligned} k_{\text{QOTP}} &\leftarrow \left(\left\{ |A_{s, s'}^{(i)}\rangle \right\}_{i \in [n]}, k_{P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}} \right) \\ \pi &\leftarrow \mathcal{S}(x, \text{crs}, \tau) \\ \pi_{\text{QOTP}} &\leftarrow (\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp, \pi) \end{aligned}$$

and sends $(k_{\text{QOTP}}, \pi_{\text{QOTP}})$ to $\mathcal{A}$.

5. $\mathcal{B}$ receives $(x_1, \mathbf{u}_1), (x_2, \mathbf{u}_2)$ from $\mathcal{A}$ and does the following:

- (a) If $(x_1, \mathbf{u}_1) = (x_2, \mathbf{u}_2)$, $\mathcal{B}$ aborts.
- (b) If $\forall j \in [2] : P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}(x_j, \mathbf{u}_j) = 1$, $\mathcal{B}$ aborts.
- (c) If $\exists j \in [2] : P_{\hat{\mathbf{A}}, \hat{\mathbf{A}}^\perp}(x_j, \mathbf{u}_j) = 0$, $\mathcal{B}$ sets the evaluation query set to be $\{(x_{2-j+1}, \mathbf{u}_{2-j+1})\}$ and the challenge query is $(x_j, \mathbf{u}_j)$.

6. $\mathcal{B}$ sends the evaluation query set $\{(x_{2-j+1}, \mathbf{u}_{2-j+1})\}$ to $\mathcal{C}$.
7. $\mathcal{C}$ sends $y_{2-j+1} = \mathcal{F}.\text{Eval}(k, (x_{2-j+1}; \mathbf{u}_{2-j+1}))$ to $\mathcal{B}$.
8. $\mathcal{B}$ sends challenge query $(x_j, \mathbf{u}_j)$ to $\mathcal{C}$.
9. $\mathcal{C}$ samples $c \xleftarrow{\$} \{0, 1\}$ and sends y_j where

$$y_j = \begin{cases} \mathcal{F}.\text{Eval}(k, (x_j; \mathbf{u}_j)) & c = 0 \\ \mathcal{D}(\{0, 1\}^m) & c = 1 \end{cases}$$

10. $\mathcal{B}$ samples $b_{2-j+1} \xleftarrow{\$} \{0, 1\}$ and sends z_1, z_2 to $\mathcal{A}$ where $z_j = y_j$ and

$$z_{2-j+1} = \begin{cases} y_{2-j+1} & b_{2-j+1} = 0 \\ \mathcal{D}(\{0, 1\}^m) & b_{2-j+1} = 1 \end{cases}$$

11. $\mathcal{A}$ sends b'_1, b'_2 to $\mathcal{B}$.
12. $\mathcal{B}$ outputs b'_j .

This reduction from Hyb_1 to decision CF security is similar to the reduction in Theorem 42 with the only exception that $\mathcal{B}$ additionally generates $\pi \leftarrow \mathcal{S}(x, \text{crs}, \tau)$ here. Using the same case-by-case analysis, we have $\text{Adv}_{\mathcal{B}}^{\text{CF}}(1^\lambda) = 1/\text{poly}(\lambda)$ which is a contradiction. Hence $\text{Adv}_{\mathcal{A}}^{\text{Hyb}_1}(1^\lambda) = \text{negl}(\lambda)$.

Finally, from lemma 49, 50 and 51, we have $\text{Adv}_{\mathcal{A}}^{\mathcal{F}_{\text{mal}} - \text{QOTP}}(1^\lambda) = \text{negl}(\lambda)$ for all QPT $\mathcal{A}$. □

8 Applications

We present new quantum one-time primitives and their construction in this section.

8.1 Secret Key Encryption

8.1.1 Preliminaries

First, we recall the definition of secret key encryption scheme.

Definition 52 (Secret Key Encryption). *A secret key encryption (SKE) functionality with plaintext space $\{\mathcal{P}_\lambda\}$ and ciphertext space $\{\mathcal{C}_\lambda\}$ is a tuple of QPT algorithms $(\text{KeyGen}, \text{Enc}, \text{Dec})$ where*

- $\text{KeyGen}(1^\lambda)$ is a PPT algorithm that takes as input security parameter λ and outputs secret key k .
- $\text{Enc}(k, m; r)$ is a PPT algorithm that takes as input secret key k , a plaintext $m \in \mathcal{P}_\lambda$, randomness r and outputs ciphertext (c, r) .
- $\text{Dec}(k, c, r)$ is a deterministic algorithm that takes as input secret key k , ciphertext (c, r) and outputs a plaintext $m' \in \mathcal{P}_\lambda$.

Additionally, it satisfies the following properties:

- **Correctness:** For all $\lambda \in \mathbb{N}$, $m \in \mathcal{P}_\lambda$ and randomness r ,

$$\Pr \left[m \leftarrow \text{Dec}(k, c, r) \mid \begin{smallmatrix} k \leftarrow \text{KeyGen}(1^\lambda) \\ (c, r) \leftarrow \text{Enc}(k, m; r) \end{smallmatrix} \right] = 1$$

- **Pseudorandom Ciphertext:** Let $\mathcal{C}_\lambda \subseteq \{0, 1\}^\ell$. For all $k \leftarrow \text{KeyGen}(1^\lambda)$ and QPT $\mathcal{A}$ with oracle access to $\text{Enc}(k, \cdot)$, we have

$$\Pr \left[b \leftarrow \mathcal{A}(c_b) \mid \begin{smallmatrix} m \leftarrow \mathcal{A}^{\text{Enc}(k, \cdot)}, b \xleftarrow{\$} \{0, 1\} \\ c_0 \leftarrow \text{Enc}(k, m; r), c_1 \xleftarrow{\$} \{0, 1\}^\ell \end{smallmatrix} \right] = \frac{1}{2} + \text{negl}(\lambda)$$

We give an instantiation of secret key encryption functionality defined above from pseudorandom functions.

Definition 53. Let $\text{PRF} : \{0, 1\}^{n'} \rightarrow \{0, 1\}^\ell$ be a pseudorandom function. Then $(\text{KeyGen}, \text{Enc}, \text{Dec})$ are defined as following:

- $\text{KeyGen}(1^\lambda) : \text{Output } k \leftarrow \text{PRF.KeyGen}(1^\lambda)$
- $\text{Enc}(k, m; r) : \text{Output } (m \oplus \text{PRF.Eval}(k, r), r)$.
- $\text{Dec}(k, c, r) : \text{Output } c \oplus \text{PRF.Eval}(k, r)$

$(\text{KeyGen}, \text{Enc}, \text{Dec})$ is a secret key encryption functionality satisfying correctness and pseudorandom ciphertext property.

8.1.2 QOTP for Secret Key Encryption

Quantum one-time program for secret key encryption will consist of QPT algorithms (**Generate**, **QEnc**) which allows a party to encrypt at most one message using the secret key (As the one-time program computes encryption, we replace **Evaluate** with **QEnc** in the one-time program in Definition 18 and everything else remains same). We require our one-time secret key encryption scheme to satisfy decision weak operational security (Definition 19) where an adversary sends $(m_1, r_1), (m_2, r_2)$ to the challenger where m_j is a message and r_j is randomness used to encrypt m . We additionally impose the condition on $\mathcal{A}$ that $m_1 \neq m_2$. The challenger samples two random bits $b_1, b_2 \xleftarrow{\$} \{0, 1\}$ and sends $c_j = \text{Enc}(k, m_j; r_j)$ if $b_j = 0$ and uniformly random c_j otherwise. The adversary guesses b_1, b_2 and the guarantee is that no efficient adversary can guess both b_1, b_2 except with negligible advantage.

We construct a quantum one-time program for secret key encryption functionality (Definition 53) **SKE – QOTP** using quantum one-time program for pseudorandom functions **PRF – QOTP** which can be realized from **LWE**.

Let $\text{PCPRF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^{n\lambda} \rightarrow \{0, 1\}^\ell\}$ be a privately constrained randomized PRF (Definition 6) that takes as input $x \in \{0, 1\}^n, u \in \{0, 1\}^{n\lambda}$ and outputs $y = f_k(x; u) \in \{0, 1\}^\ell$ for $\ell = \text{poly}(\lambda)$. Let **PCPRF – QOTP** be a quantum one-time program for **PCPRF** instantiated from Section 5.3. Let $\mathcal{P} = \mathcal{C} = \{0, 1\}^\ell$. Then the **QOTP** scheme for encryption (**Generate**, **QEnc**) will be

- **Generate** $(1^\lambda, k)$:

1. Output $k_{\text{QOTP}} \leftarrow \text{PCPRF} - \text{QOTP.Generate}(1^\lambda, k)$.
- $\text{QEnc}(k_{\text{QOTP}}, m)$:

On input QOTP key k_{QOTP} and plain text $m \in \mathcal{P}_\lambda$ do

1. Samples $x \xleftarrow{\$} \{0, 1\}^n$.
2. Compute $(y, u) \leftarrow \text{PCPRF.Evaluate}(k_{\text{QOTP}}, x)$.
3. Output $(m \oplus y, r)$ where $r = (x, u)$.

Theorem 54. *If PCPRF satisfies decision PCF security, the quantum one-time scheme SKE – QOTP is a quantum one-time program for secret key encryption functionality (Definition 53) satisfying correctness (Definition 18) and decision weak operational security (Definition 19).*

Proof. Correctness. By the correctness of QOTP for pseudorandom functions, output of QEnc is $(c, r) = (m \oplus \text{PCPRF.Eval}(k, r), r)$ where $r = (x, u)$ is uniformly random. Hence $\forall k \leftarrow \text{PCPRF.KeyGen}(1^\lambda), m \in \mathcal{P}_\lambda$, we have

$$\Pr \left[(c, r) = \text{Enc}(k, m; r) \mid \begin{matrix} k_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k) \\ (c, r) \leftarrow \text{QEnc}(k_{\text{QOTP}}, m) \end{matrix} \right] = 1$$

and correctness is satisfied.

Security. Since PCPRF satisfy decision PCF security (Theorem 41), PCPRF – QOTP satisfies decision weak operational security (Theorem 41). Suppose there is a QPT $\mathcal{A}$ such that can win decision weak operational security game with non-negligible advantage i.e. $\text{Adv}_{\mathcal{A}}^{\text{SKE-QOTP}}(1^\lambda) = 1/p(\lambda)$ for polynomial $p(\cdot)$. We construct QPT $\mathcal{B}$ that uses $\mathcal{A}$ in a black-box way such that $\text{Adv}_{\mathcal{B}}^{\text{PCPRF-QOTP}}(1^\lambda)$ is non-negligible which is a contradiction.

Description of $\mathcal{B}$:

1. $\mathcal{C}$ samples $k \leftarrow \text{PCPRF.KeyGen}(1^\lambda)$ and sends $k_{\text{QOTP}} \leftarrow \text{PCPRF} - \text{QOTP.Generate}(1^\lambda, k)$ to $\mathcal{B}$.
2. $\mathcal{B}$ sends k_{QOTP} to $\mathcal{A}$.
3. $\mathcal{B}$ receives (m_1, r_1) and (m_2, r_2) from $\mathcal{A}$. $\mathcal{B}$ aborts if $r_1 = r_2$.
4. $\mathcal{B}$ sends $r_1 = (x_1, u_1), r_2 = (x_2, u_2)$ to $\mathcal{C}$.
5. $\mathcal{C}$ samples $b_1, b_2 \xleftarrow{\$} \{0, 1\}$ and sends $y_1, y_2 \in \{0, 1\}^\ell$ to $\mathcal{B}$ such that
$$y_j = \begin{cases} \text{PCPRF.Eval}(k, (x_j; u_j)) & b_j = 0 \\ U(\{0, 1\}^\ell) & b_j = 1 \end{cases}$$
6. $\mathcal{B}$ sends $(c_1, r_1), (c_2, r_2)$ to $\mathcal{A}$ where $c_j = m_j \oplus y_j$.
7. $\mathcal{A}$ sends b'_1, b'_2 to $\mathcal{B}$.
8. $\mathcal{B}$ outputs b'_1, b'_2 .

Since, $\mathcal{A}$ wins with probability $\frac{1}{2} + \frac{1}{p(\lambda)}$, $\Pr[\mathcal{B} \text{ doesn't abort}] \geq \frac{1}{2} + \frac{1}{\text{poly}(\lambda)}$. If $\mathcal{C}$ samples $b_j = 0$, then $y_j = \text{PCPRF.Eval}(k, (x_j; u_j))$ and $c_j = m_j \oplus y_j$. If $b_j = 1$, then $y_j \xleftarrow{\$} \{0, 1\}^\ell$ and $c_j \xleftarrow{\$} \{0, 1\}^\ell$. Thus $\text{Adv}_{\mathcal{B}}^{\text{PCPRF-QOTP}}(1^\lambda) = \text{Adv}_{\mathcal{A}}^{\text{SKE-QOTP}}(1^\lambda) = 1/p(\lambda)$. $\square$

8.2 Message Authentication Code

8.2.1 Preliminaries

We recall the definition of message authentication codes.

Definition 55. A message authentication code (MAC) functionality with message space $\{\mathcal{M}_\lambda\}$ and tag space $\{\mathcal{T}_\lambda\}$ is a tuple of algorithms $(\text{KeyGen}, \text{MAC}, \text{Verify})$ where:

- $\text{KeyGen}(1^\lambda)$ is a PPT algorithm that takes as input security parameter λ and outputs secret key k .
- $\text{MAC}(k, m; r)$ is a PPT algorithm that takes as input secret key k , a message $m \in \mathcal{M}_\lambda$, randomness r and (τ, m, r) where $\tau \in \mathcal{T}_\lambda$.
- $\text{Verify}(k, \tau, m; r)$ is a PPT algorithm that takes as input secret key k , a tag $\tau \in \mathcal{T}_\lambda$, a message $m \in \mathcal{M}_\lambda$, randomness r and outputs $\top/\perp$.

Additionally, it satisfies the following properties:

- **Correctness:** For all $\lambda \in \mathbb{N}, m \in \mathcal{M}_\lambda$ and randomness r , we have

$$\Pr \left[\top \leftarrow \text{Verify}(k, m, \tau; r) \mid \begin{array}{l} k \leftarrow \text{KeyGen}(1^\lambda) \\ (\tau, m, r) \leftarrow \text{MAC}(k, m; r) \end{array} \right] = 1$$

- **Unforgeability:** For all QPT $\mathcal{A}$ with access to MAC oracle,

$$\Pr \left[\top \leftarrow \text{Verify}(k, m^*, \tau^*) \wedge m^* \notin Q \mid \begin{array}{l} k \leftarrow \text{KeyGen}(1^\lambda) \\ (m^*, \tau^*) \leftarrow \mathcal{A}^{\text{MAC}(k, \cdot)} \end{array} \right] \leq \text{negl}(\lambda)$$

where Q is the set of messages on which $\mathcal{A}$ queries the MAC oracle.

We give an instantiation of MAC functionality as defined above from pseudorandom functions.

Definition 56. Let $\text{PRF} : \{0, 1\}^{n'} \rightarrow \{0, 1\}^\ell$ be a pseudorandom functions. Then $(\text{KeyGen}, \text{MAC}, \text{Verify})$ are defined as following:

- $\text{KeyGen}(1^\lambda) : \text{Output } k \leftarrow \text{PRF.KeyGen}(1^\lambda)$.
- $\text{MAC}(k, m; r) : \text{Output } (\tau, m, r) \text{ where } \tau = \text{PRF.Eval}(k, (m; r))$
- $\text{Verify}(k, \tau, m; r) : \text{Output } \top \text{ if } \tau = \text{PRF.Eval}(k, (m; r)), \text{ output } \perp \text{ otherwise.}$

$(\text{KeyGen}, \text{MAC}, \text{Verify})$ is a MAC functionality satisfying correctness and unforgeability.

8.2.2 QOTP for Message Authentication Code

Quantum one-time program for message authentication code will consist of $(\text{Generate}, \text{QMAC})$ allows a party to compute MAC of at most one message while it can verify multiple messages (Similar to one-time SKE, we replace Evaluate with QMAC as the one-time program corresponds to computing MAC). Therefore, we require our one-time MAC to satisfy search weak operational security (Definition 20). However, we show that the one-time program satisfies the stronger weak operational security (Definition 19) where the adversary sends $(m_1, r_1), (m_2, r_2)$ and has to distinguish

the tag $\tau_j \leftarrow \text{MAC}(k, m_j; r_j)$ from uniformly random string $\tau_j \xleftarrow{\$} \mathcal{T}_\lambda$.

We construct a quantum, one-time program for MAC functionality (Definition 56) MAC-QOTP using quantum one-time program for pseudorandom functions PRF-QOTP which can be instantiated from LWE.

Let $\text{PCPRF} = \{f_k : \{0, 1\}^n \times \{0, 1\}^{n\lambda} \rightarrow \{0, 1\}^\ell\}$ be a privately constrained randomized PRF (Definition 6) that takes as input $x \in \{0, 1\}^n, u \in \{0, 1\}^{n\lambda}$ and outputs $y = f_k(x; u) \in \{0, 1\}^\ell$. Let PCPRF-QOTP be a quantum one-time program for PCPRF instantiated from Section 3.2. By using PCPRF-QOTP as a subroutine, we can construct a quantum one-time program for message authentication MAC-QOTP with $\mathcal{M} = \{0, 1\}^n$ and $\mathcal{T} = \{0, 1\}^\ell$ as follows:

- Generate($1^\lambda, k$): Output $k_{\text{QOTP}} \leftarrow \text{PCPRF-QOTP.Generate}(1^\lambda, k)$.
- QMAC(k_{QOTP}, m):

On input $k_{\text{QOTP}}, m \in \{0, 1\}^n$ do

1. Generate $(y, r) \leftarrow \text{PCPRF-QOTP.Evaluate}(k_{\text{QOTP}}, m)$.
2. Output (τ, m, r) where .

Theorem 57. *If PCPRF satisfies decision PCF security, the quantum one-time scheme MAC-QOTP is a quantum one-time program for message authentication code functionality satisfying correctness (Definition 18) and decision weak operational security (Definition 19).*

Proof. Correctness. By the correctness of PCPRF-QOTP, output of QMAC is $(\tau, m, r) = (\text{PCPRF.Eval}(k, (m; r)), m, r)$ where $r \in \{0, 1\}^{n\lambda}$ is uniformly random. Hence $\forall k \leftarrow \text{PCPRF.KeyGen}(1^\lambda), m \in \mathcal{M}$, we have

$$\Pr \left[(\tau, m, r) = \text{MAC}(k, m; r) \mid \begin{matrix} k_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k) \\ (\tau, m, r) \leftarrow \text{QMAC}(k_{\text{QOTP}}, m) \end{matrix} \right] = 1$$

and correctness is satisfied.

Security. Since PCPRF satisfy decision PCF security (Theorem 41), PCPRF-QOTP satisfies decision weak operational security. Suppose there is a QPT $\mathcal{A}$ that such that $\text{Adv}_{\mathcal{A}}^{\text{MAC-QOTP}}(1^\lambda) = 1/p(\lambda)$ for polynomial $p(\cdot)$. We construct a QPT $\mathcal{B}$ that uses $\mathcal{A}$ in a black-box way such that $\text{Adv}_{\mathcal{B}}^{\text{PCPRF-QOTP}}(1^\lambda)$ is non-negligible.

Description of $\mathcal{B}$:

1. $\mathcal{C}$ samples $k \leftarrow \text{PCPRF.KeyGen}(1^\lambda)$ and sends $k_{\text{QOTP}} \leftarrow \text{PCPRF-QOTP.Generate}(1^\lambda, k)$ to $\mathcal{B}$.
2. $\mathcal{B}$ sends k_{QOTP} to $\mathcal{A}$.
3. $\mathcal{B}$ receives (m_1, r_1) and (m_2, r_2) from $\mathcal{A}$. $\mathcal{B}$ aborts if $(m_1, r_1) = (m_2, r_2)$.
4. $\mathcal{B}$ sends $(m_1, r_1), (m_2, r_2)$ to $\mathcal{C}$.
5. $\mathcal{C}$ samples $b_1, b_2 \xleftarrow{\$} \{0, 1\}$ and sends $y_1, y_2 \in \{0, 1\}^\ell$ to $\mathcal{B}$ such that

$$y_j = \begin{cases} \text{PCPRF.Eval}(k, (x_j; r_j)) & b_j = 0 \\ U(\{0, 1\}^\ell) & b_j = 1 \end{cases}$$

6. $\mathcal{B}$ sends $(\tau_1, m_1, r_1), (\tau_2, m_2, r_2)$ to $\mathcal{A}$ where $\tau_j = y_j$.

7. $\mathcal{B}$ receives $b'_1, b'_2 \in \{0, 1\}$ from $\mathcal{A}$.

8. $\mathcal{B}$ outputs b'_1, b'_2 .

Since $\mathcal{A}$ wins with probability $\frac{1}{2} + \frac{1}{p(\lambda)}$, $\Pr[\mathcal{B} \text{ doesn't abort}] \geq \frac{1}{2} + \frac{1}{p(\lambda)}$. For $j \in [2]$: if $\mathcal{C}$ samples $b_j = 0$, $\tau_j = y_j = \text{PCPRF.Eval}(k, (m_j; r_j))$. If $b_j = 1$, $\tau_j = y_j \xleftarrow{\$} \{0, 1\}^\ell$. Thus, $\text{Adv}_{\mathcal{B}}^{\text{PCPRF-QOTP}}(1^\lambda) = \text{Adv}_{\mathcal{A}}^{\text{MAC-QOTP}}(1^\lambda) = 1/p(\lambda)$. $\square$

8.3 One-Message 2PC

We study the feasibility of malicious secure one-message two-party computation (2PC) of classical functionalities. In the classical setting, even semi-honest one-message 2PC of any functionality is impossible but the positive results for quantum one-time program motivates us to ask whether one-message 2PC of any classical functionality is feasible with quantum resources. To this end, we define a notion of one-message malicious-secure 2PC which is impossible to realize for any classical functionality using classical resources. Then, we give a positive result that there are certain classical functionalities for which such a 2PC can be realized using quantum resources.

We consider the following setting for one-message 2PC of a randomized functionality g : There are two (malicious) parties P_1 (sender) and P_2 (receiver) with private inputs x_1, x_2 . P_1 sends a message m to P_2 . If m is maliciously generated, P_2 outputs $\perp$ otherwise P_2 outputs $g(x_1, x_2; r)$ for uniformly random r . This is quite similar to the quantum one-time programs where sender generates a QOTP program and sends to evaluator who can evaluate the program at input of its choice. Note that since quantum one-time program for deterministic functionalities is impossible, one-message 2PC of deterministic functions is impossible even with quantum resources. For security, we ideally want both the parties to be simulatable. However, reaching this level of security for P_2 would entail constructing a one-time program for a related functionality with simulation security in plain model, so imposing such a condition may be overly ambitious. We define a malicious secure one-message 2PC in a way that has security guarantee that is both meaningful and still achievable. We require (malicious) P_2 to satisfy search weak operational security (Definition 20) so that P_2 can learn at most one output of $g(x_1, \cdot)$ and (malicious) P_1 still satisfies simulation based security. This is impossible to achieve using classical communication for any functionality since P_2 can always make copies of message m and learn any number of outputs of $g(x_1, \cdot)$ violating the operational security.

We begin by defining the ideal functionality Ideal_g (Figure 1) for 2PC of randomized function $g : \{0, 1\}^{n_1} \times \{0, 1\}^{n_2} \times \{0, 1\}^\ell \rightarrow \{0, 1\}^m$. The ideal experiment is as follows: P_1, P_2 send their inputs x_1, x_2 to Ideal_g respectively. If P_1 inputs $\perp$, Ideal_g sends $\perp$ to P_2 . Otherwise Ideal_g samples a random string $r \in \{0, 1\}^\ell$ and sends $g(x_1, x_2; r)$ to P_2 .

Definition 58 (One-message Malicious-secure One-sided 2PC). *Let $g : \{0, 1\}^{n_1} \times \{0, 1\}^{n_2} \times \{0, 1\}^\ell \rightarrow \{0, 1\}^m$ be a randomized function. A one-message malicious-secure one-sided 2PC is the set of algorithm (Setup, Send, Recv) where*

- **Setup**(1^λ) is a QPT algorithm that takes as input security parameter λ and outputs crs .
- **Send**(x_1, crs) is a QPT algorithm that takes as input $x_1 \in \{0, 1\}^{n_1}, \text{crs}$ and outputs message m .

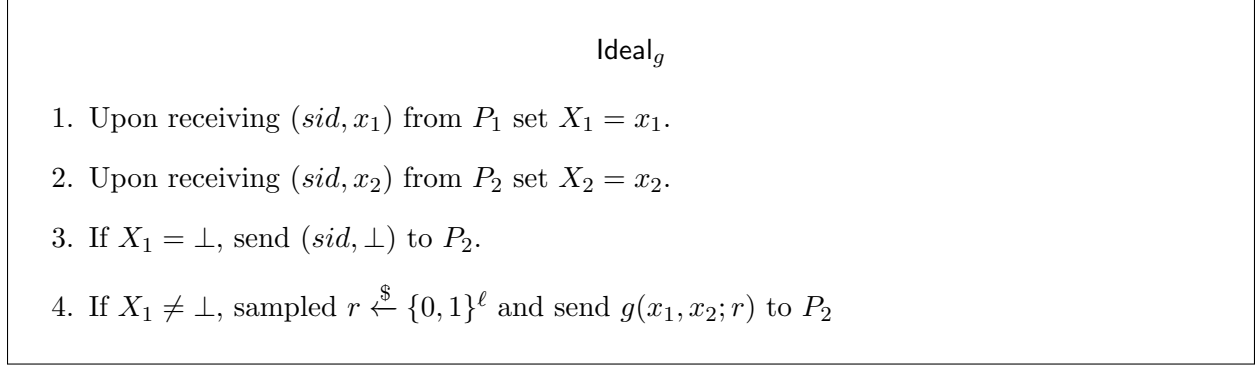


Figure 1: Ideal Functionality Ideal_g

- $\text{Recv}(x_2, m)$ is a QPT algorithm that takes as input $x_2 \in \{0, 1\}^{n_2}$, message m and outputs $\perp$ or $y \in \{0, 1\}^m$.

Additionally, it satisfies following properties:

- **Correctness:** For all $x_1 \in \{0, 1\}^{n_1}, x_2 \in \{0, 1\}^{n_2}, \exists r \in \{0, 1\}^\ell$ we have

$$\Pr \left[\text{Recv}(x_2, m) = g(x_1, x_2; r) \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ m \leftarrow \text{Send}(x_1, \text{crs}) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

and the following distributions are computationally indistinguishable:

$$\{\text{Recv}(x_2, \text{Send}(x_1, \text{crs}))\} \approx_c \left\{ g(x_1, x_2; r) : r \xleftarrow{\$} \{0, 1\}^\ell \right\}$$

- **Simulation Security against malicious sender:** For all QPT $\mathcal{A}$ corrupting sender, there is a QPT simulator $\mathcal{S}$ such that

$$\left(\text{View}_{\mathcal{A}}^{\text{Real}}, \text{Out}^{\text{Real}} \right) \approx_c \left(\text{View}_{\mathcal{S}}^{\text{Ideal}}, \text{Out}^{\text{Ideal}} \right)$$

where $(\text{View}_{\mathcal{A}}^{\text{Real}}, \text{Out}^{\text{Real}})$ is the joint distribution of view of $\mathcal{A}$ and output of honest receiver in real execution of 2PC and $(\text{View}_{\mathcal{S}}^{\text{Ideal}}, \text{Out}^{\text{Ideal}})$ is the joint distribution of view of $\mathcal{S}$ and output of honest receiver in the ideal execution.

- **Weak Operational Security against malicious receiver:** For all $x_1 \in \{0, 1\}^{n_1}$ and $m \leftarrow \text{Send}(x_1, \text{crs})$, no QPT $\mathcal{A}$ can output $(x_2, r, y), (x'_2, r', y')$ such that $(x_2, r) \neq (x'_2, r')$ and $y = g(x_1, x_2; r), y' = g(x_1, x'_2; r')$ except with negligible probability:

$$\Pr \left[g(x_1, x_2; r) = y \wedge g(x_1, x'_2; r') = y' \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda), m \leftarrow \text{Send}(x_1, \text{crs}) \\ (x_2, r, y), (x'_2, r', y') \leftarrow \mathcal{A}(m, \text{crs}) \\ (x_2, r, y) \neq (x'_2, r', y') \end{array} \right] = \text{negl}(\lambda)$$

Let $\mathcal{F} = \{f_k : \{0, 1\}^n \times \{0, 1\}^{n_\lambda} \rightarrow \{0, 1\}^m\}$ be a randomized constrained functionality (Definition 22) satisfying search weak operational security (Definition 25). Let CosetSamp and VecSamp be two algorithms as follows:

On input $x_1 \in \{0, 1\}^{\lambda+p}$, $x_2 \in \{0, 1\}^n$, randomness $r \in \{0, 1\}^{n\ell}$ do:

1. Parse $x_1 = (k, z) \in \{0, 1\}^\lambda \times \{0, 1\}^p$.
2. Compute $x_2^{(1)} x_2^{(2)} \dots x_n^{(n)} \leftarrow \text{Decomp}(x_2)$.
3. Parse $(r_1, r_2, \dots, r_n) \leftarrow r$ where $\forall i : r_i \in \{0, 1\}^\ell$.
4. Compute $\mathbf{A}, \mathbf{s}, \mathbf{s}' \leftarrow \text{CosetSamp}(z)$.
5. For all $i \in [n]$, compute $u^{(i)}$ where

$$u^{(i)} \leftarrow \begin{cases} \text{VecSamp}(A_i + s_i, r_i) & x_2^{(i)} = 0 \\ \text{VecSamp}(A_i^\perp + s'_i, r_i) & x_2^{(i)} = 1 \end{cases}$$

6. Let $\mathbf{u} = (u^{(1)}, \dots, u^{(n)}) \in \{0, 1\}^{n\lambda}$.
7. Output $g_{\mathcal{F}}(x_1, x_2; r) \leftarrow (f_k(x_2; \mathbf{u}), \mathbf{u})$

Figure 2: Description of randomized function $g_{\mathcal{F}}$

- $\text{CosetSamp}(z)$ defined in Section 7 is an efficient classical algorithm that takes as input randomness $z \in \{0, 1\}^p$ for some $p = \text{poly}(n, \lambda)$ and outputs $\mathbf{A}, \mathbf{s}, \mathbf{s}'$ such that $\{A_i + s_i, A_i^\perp + s'_i\}_{i \in [n]}$ are $2n$ cosets in $\mathbb{F}_2^\lambda$ and $\dim(A_i) = \lfloor \lambda/2 \rfloor$.
- $\text{VecSamp}(A + s, r)$ is an efficient classical algorithm that takes as input description of a coset $A + s$ and randomness $r \in \{0, 1\}^\ell$ and outputs a vector $\mathbf{u} \in A + s$. For $r \xleftarrow{\$} \{0, 1\}^\ell$, $\mathbf{u}$ is uniformly random in $A + s$.

We describe a randomized function $g_{\mathcal{F}} : \{0, 1\}^{\lambda+p} \times \{0, 1\}^n \times \{0, 1\}^{n\ell} \rightarrow \{0, 1\}^m \times \{0, 1\}^{n\lambda}$ in Figure 2. Next, we show that there is a 2PC satisfying Definition 58 of $g_{\mathcal{F}}$ in the following theorem.

Theorem 59. *If $\mathcal{F} = \{f_k : \{0, 1\}^n \times \{0, 1\}^{n\lambda} \rightarrow \{0, 1\}^m\}$ is a constrained randomized functionality satisfying search CF security (Definition 25). Assuming quantum secure indistinguishability obfuscator, injective one-way functions and NIZK-PoK, there is a one-message malicious-secure one-sided 2PC of function $g_{\mathcal{F}}$ defined in Figure 2.*

Proof. Since $\mathcal{F}$ satisfies search CF security. $\mathcal{F}_{\text{mal}} - \text{QOTP}$ from Section 7.1 is a malicious-secure QOTP (Theorem 47) satisfying search weak operational security (Definition 20). The 2PC protocol for function $g_{\mathcal{F}}$ will be as follows:

- **Setup**(1^λ) :
 1. Output $\text{crs} \leftarrow \mathcal{F}_{\text{mal}} - \text{QOTP}.\text{Setup}(1^\lambda)$.
- **Send**(x_1, crs) :
 1. Parse $x_1 = (k, z) \in \{0, 1\}^\lambda \times \{0, 1\}^p$.

2. Compute $(k_{\text{QOTP}}, \pi_{\text{QOTP}}) \leftarrow \mathcal{F}_{\text{mal}} - \text{QOTP.Generate}(1^\lambda, k, \text{crs}; z)$ where the cosets $\{A_i + s_i, A_i^\perp + s'_i\}_{i \in [n]}$ are generated using $\text{CosetSamp}(z)$ in the **Generate** algorithm.
3. Output $m = (k_{\text{QOTP}}, \pi_{\text{QOTP}})$.

• $\text{Recv}(m, x_2, \text{crs}) :$

1. Parse $m = (k_{\text{QOTP}}, \pi_{\text{QOTP}})$.
2. Compute $(b, \rho_{\text{QOTP}}) \leftarrow \mathcal{F}_{\text{mal}} - \text{QOTP.Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs})$.
3. If $b = \perp$, output $\perp$.
4. Output $(y, u) \leftarrow \mathcal{F}_{\text{mal}} - \text{QOTP.Evaluate}(k_{\text{QOTP}}, x_2)$

Correctness. Suppose $m \leftarrow \text{Send}(x_1, \text{crs})$ and let $(y, \mathbf{u}) \leftarrow \text{Recv}(x_2, \text{crs})$. Then from correctness of $\mathcal{F}_{\text{mal}} - \text{QOTP}$ (Theorem 47), $(y, \mathbf{u}) = (f_k(x_2, \mathbf{u}), \mathbf{u}) = g(x_1, x_2; r)$ for $r \xleftarrow{\$} \{0, 1\}^{n_\ell}$ and we have for all $x_1 \in \{0, 1\}^{\lambda+p}$, $x_2 \in \{0, 1\}^{n_2}$, $\exists r \in \{0, 1\}^{n_\ell}$:

$$\Pr \left[\text{Recv}(x_2, m) = g(x_1, x_2; r) \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ m \leftarrow \text{Send}(x_1, \text{crs}) \end{array} \right] = 1$$

$$\{\text{Recv}(x_2, \text{Send}(x_1, \text{crs}))\} \approx_c \left\{ g(x_1, x_2; r) : r \xleftarrow{\$} \{0, 1\}^{n_\ell} \right\}$$

Weak Operational Security. Suppose there is a QPT $\mathcal{A}$ that outputs $(x_2, r, y), (x'_2, r', y')$ with non-negligible advantage. Then, there is a QPT $\mathcal{B}$ that wins search weak operational security for $\mathcal{F}_{\text{mal}} - \text{QOTP}$ (Definition 20) with same advantage.

Description of $\mathcal{B}$:

1. Challenger $\mathcal{C}$ samples $\text{crs} \leftarrow \text{Setup}(1^\lambda)$ and sends crs to $\mathcal{B}$.
2. $\mathcal{C}$ generates $k \leftarrow \mathcal{F}.\text{KeyGen}(1^\lambda)$.
3. $\mathcal{C}$ sends $(k_{\text{QOTP}}, \pi_{\text{QOTP}}) \leftarrow \mathcal{F}_{\text{mal}} - \text{QOTP.Generate}(1^\lambda, k)$ to $\mathcal{B}$.
4. $\mathcal{B}$ sends $m = (k_{\text{QOTP}}, \pi_{\text{QOTP}})$ to $\mathcal{A}$.
5. $\mathcal{A}$ sends $(x_2, r, y), (x'_2, r', y')$.
6. $\mathcal{C}$ outputs $(x_2, y), (x'_2, y')$ to $\mathcal{C}$.

If $\mathcal{A}$ is successful, then $y = (f_k(x_2; \mathbf{u}), \mathbf{u})$ and $y' = (f_k(x'_2; \mathbf{u}'), \mathbf{u}')$. Then, $(x_2, y) = (x_2, \mathbf{u}, f_k(x_2; \mathbf{u}))$, $(x'_2, y') = (x'_2, \mathbf{u}', f_k(x'_2; \mathbf{u}'))$ and $(x_2, \mathbf{u}) \neq (x'_2, \mathbf{u}')$. Thus, $\mathcal{B}$ wins with same probability.

Simulation Security against Malicious Sender. Let $\mathcal{A}$ be a QPT adversary corrupting the sender. We describe a QPT simulator $\mathcal{S}$ such that joint distribution of view of $\mathcal{A}$ and output of honest receiver in real execution of 2PC is indistinguishable from joint distribution of view of $\mathcal{S}$ and output of honest receiver in ideal execution with ideal functionality. Let $E = (E_1, E_2)$ be the QPT key extractor defined in Definition 45. The extractor additionally outputs $z \in \{0, 1\}^p$ which was used by generator to sample coset states $\mathbf{A}, \mathbf{s}, \mathbf{s}' \leftarrow \text{CosetSamp}(z)$ in **Generate** algorithm. Note that

in the description of E_2 (Theorem 47), it is already extracting z from NIZK proof π but doesn't output it.

Description of $\mathcal{S}$:

1. Generate $(\text{crs}, \tau) \leftarrow E_1(1^\lambda)$.
2. Run $m \leftarrow \mathcal{A}(\text{crs})$.
3. Parse $m = (k_{\text{QOTP}}, \pi_{\text{QOTP}})$ and run $(b, \rho_{\text{QOTP}}) \leftarrow \mathcal{F}_{\text{mal}} - \text{QOTP.Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs})$.
4. If $b = \perp$, send $\perp$ to the ideal functionality.
5. If $b = \top$, run $(k, z) \leftarrow E_2(\rho_{\text{QOTP}}, \pi_{\text{QOTP}}, \text{crs}, \tau)$.
6. Send $x_1 = (k, z)$ to the ideal functionality.

□

By the key extraction property of $\mathcal{F}_{\text{mal}} - \text{QOTP}$ (Definition 45), crs generated by $\text{Setup}(1^\lambda)$ and $E_1(1^\lambda)$ are computationally indistinguishable.

Suppose $\mathcal{A}$ sends a honestly generated message m . In this case, $\mathcal{F}_{\text{mal}} - \text{QOTP.Verify}(k_{\text{QOTP}}, \pi_{\text{QOTP}})$ always outputs $(\top, k_{\text{QOTP}})$. E_2 outputs (k, z) such that $k_{\text{QOTP}} \leftarrow \mathcal{F}_{\text{mal}} - \text{QOTP.Generate}(1^\lambda, k; z)$ except with negligible probability. $\mathcal{S}$ sends $x_1 = (k, z)$ to the ideal functionality which then sends $g(x_1, x_2; r) = (f_k(x_2; \mathbf{u}), \mathbf{u})$ to receiver where $\mathbf{u} \xleftarrow{\$} \{0, 1\}^{n\lambda}$ as $r \xleftarrow{\$} \{0, 1\}^{n\ell}$. Since $k_{\text{QOTP}} \leftarrow \mathcal{F}_{\text{mal}} - \text{QOTP.Generate}(1^\lambda, k; z)$ so $\text{Recv}(k_{\text{QOTP}}, x_2, \text{crs})$ outputs $f_k(x_2; \mathbf{u}), \mathbf{u}$ for uniformly random $\mathbf{u}$. Therefore, the joint distribution of view and output is computationally indistinguishable in the real and ideal experiment.

Now suppose $\mathcal{A}$ sends a corrupted message $m = (k_{\text{QOTP}}, \pi_{\text{QOTP}})$. There are two cases: (i) the obfuscated coset membership circuits and the constrained key $\{\hat{\mathbf{T}}, \hat{\mathbf{V}}, k_C\}$ are corrupted, and (ii) coset states $\{\psi_i\}_i$ are corrupted.

- If $\{\hat{\mathbf{T}}, \hat{\mathbf{V}}, k_C\}$ is corrupted, then by the extraction property of NIZK-PoK (Definition 13), Verify will output $\perp$ except with negligible probability. So $\mathcal{S}$ sends $\perp$ to ideal functionality and output of receiver is $\perp$. In the real execution, receiver runs $\text{Recv}(m, \text{crs})$ which will runs Verify as a subroutine and output $\perp$ except with negligible probability. Thus, the distribution is of view and output is computationally indistinguishable in the real and ideal experiment.
- Suppose the coset states $\{\psi_i\}_i$ are corrupted and $\{\hat{\mathbf{T}}, \hat{\mathbf{V}}, k_C\}$ is honestly generated. If the amplitude of state $A_{s,s'}^{(i)}$ in $|\psi_i\rangle$ is $\sqrt{q}$, then Verify projects onto the coset state and outputs $\top$ with probability q and $\perp$ with probability $1 - q$. Moreover, if Verify outputs $\top$, the residual state is indeed the coset state $|A_{s,s'}^{(i)}\rangle$ post verification and receiver outputs $(f_k(x_2; \mathbf{u}), \mathbf{u})$ in both real and ideal experiments (from key extracting property of $\mathcal{F}_{\text{mal}} - \text{QOTP}$, if verification passes then E_2 outputs (k, z) such that $\rho_{\text{QOTP}} \leftarrow \text{Generate}(1^\lambda, k; z)$). So in real experiment, honest receiver outputs $\perp$ with probability $(1 - q)$ or $g(x_1, x_2; r)$ for $r \xleftarrow{\$} \{0, 1\}^{n\ell}$ with probability q . In the ideal experiment, honest receiver outputs $\perp$ with probability $(1 - q)$ or $g(x_1, x_2; r)$ for $r \xleftarrow{\$} \{0, 1\}^{n\ell}$ with probability q . Hence, the distribution of view and output is computationally indistinguishable in real and ideal experiment.

From Theorem 59 and the impossibility of one-message 2PC using classical resources, we have the following corollary.

Corollary 60. *Assuming post-quantum security of indistinguishability obfuscator, injective one-way functions are NIZK-PoK, there is a class of randomized functionality $\{g_{\mathcal{F}}\}$ such that:*

- *There does not exist a one-message 2PC for $g_{\mathcal{F}}$ using classical resources.*
- *There exists a one-message 2PC for $g_{\mathcal{F}}$ using quantum resources.*

Acknowledgements

PA and DB are supported by the National Science Foundation under the grants FET-2329938, CAREER-2341004 and, FET-2530160.

References

- [Aar09] Scott Aaronson. “Quantum copy-protection and quantum money”. In: *2009 24th Annual IEEE Conference on Computational Complexity*. IEEE. 2009, pp. 229–242 (cit. on pp. 4, 13).
- [AB24] Prabhanjan Ananth and Amit Behera. “A modular approach to unclonable cryptography”. In: *CRYPTO*. 2024 (cit. on p. 13).
- [AC12] Scott Aaronson and Paul Christiano. “Quantum money from hidden subspaces”. In: *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*. 2012, pp. 41–60 (cit. on p. 36).
- [AJJM22] Prabhanjan Ananth, Abhishek Jain, Zhengzhong Jin, and Giulio Malavolta. “Pre-constrained encryption”. In: *13th Innovations in Theoretical Computer Science Conference*. 2022 (cit. on p. 5).
- [AKN24] Shweta Agrawal, Simran Kumari, and Ryo Nishimaki. “Don’t Trust Setup! New Directions in Pre-Constrained Cryptography”. In: *Cryptology ePrint Archive* (2024) (cit. on p. 5).
- [AL21] Prabhanjan Ananth and Rolando L La Placa. “Secure software leasing”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2021, pp. 501–530 (cit. on p. 13).
- [AP20] Shweta Agrawal and Alice Pellet-Mary. “Indistinguishability Obfuscation Without Maps: Attacks and Fixes for Noisy Linear FE”. In: *39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings*. Vol. 12105. Lecture Notes in Computer Science. Springer, 2020. DOI: 10.1007/978-3-030-45721-1_5 (cit. on p. 15).
- [BGI+01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil Vadhan, and Ke Yang. “On the (Im)possibility of Obfuscating Programs”. In: *Advances in Cryptology — CRYPTO 2001*. Ed. by Joe Kilian. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1–18. ISBN: 978-3-540-44647-7 (cit. on p. 15).

- [BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. “Functional signatures and pseudo-random functions”. In: *International workshop on public key cryptography*. Springer. 2014, pp. 501–519 (cit. on p. 5).
- [BGMZ18] James Bartusek, Jiaxin Guan, Fermi Ma, and Mark Zhandry. “Return of GGH15: Provable Security Against Zeroizing Attacks”. In: *Theory of Cryptography: 16th International Conference, TCC 2018, Panaji, India, November 11–14, 2018, Proceedings, Part II*. Panaji, India: Springer-Verlag, 2018, pp. 544–574. ISBN: 978-3-030-03809-0. DOI: 10.1007/978-3-030-03810-6_20. URL: https://doi.org/10.1007/978-3-030-03810-6_20 (cit. on p. 15).
- [BGS13] Anne Broadbent, Gus Gutoski, and Douglas Stebila. “Quantum One-Time Programs”. In: *Advances in Cryptology – CRYPTO 2013*. Springer Berlin Heidelberg, 2013, pp. 344–360. ISBN: 9783642400841. DOI: 10.1007/978-3-642-40084-1_20. URL: http://dx.doi.org/10.1007/978-3-642-40084-1_20 (cit. on pp. 4, 7, 18).
- [BLW17] Dan Boneh, Kevin Lewi, and David J Wu. “Constraining pseudorandom functions privately”. In: *IACR International Workshop on Public Key Cryptography*. Springer. 2017, pp. 494–524 (cit. on p. 5).
- [BS23] Shalev Ben-David and Or Sattath. “Quantum Tokens for Digital Signatures”. English. In: *Quantum 7* (Jan. 2023). Publisher Copyright: © 2023 Verein zur Forderung des Open Access Publizierens in den Quantenwissenschaften. All Rights Reserved. ISSN: 2521-327X. DOI: 10.22331/q-2023-01-19-901 (cit. on pp. 6, 13).
- [BSS21] Amit Behera, Or Sattath, and Uriel Shinar. “Noise-tolerant quantum tokens for MAC”. In: *arXiv preprint arXiv:2105.05016* (2021) (cit. on p. 13).
- [BTVW17] Zvika Brakerski, Rotem Tsabary, Vinod Vaikuntanathan, and Hoeteck Wee. “Private Constrained PRFs (and More) from LWE”. In: *Theory of Cryptography: 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12–15, 2017, Proceedings, Part I*. Baltimore, USA: Springer-Verlag, 2017, pp. 264–302. ISBN: 978-3-319-70499-9. DOI: 10.1007/978-3-319-70500-2_10. URL: https://doi.org/10.1007/978-3-319-70500-2_10 (cit. on pp. 5, 6, 13, 14, 30).
- [BV15] Zvika Brakerski and Vinod Vaikuntanathan. “Constrained key-homomorphic PRFs from standard lattice assumptions: Or: How to secretly embed a circuit in your PRF”. In: *Theory of Cryptography: 12th Theory of Cryptography Conference, TCC 2015, Warsaw, Poland, March 23–25, 2015, Proceedings, Part II 12*. Springer. 2015, pp. 1–30 (cit. on p. 5).
- [BW13] Dan Boneh and Brent Waters. “Constrained pseudorandom functions and their applications”. In: *Advances in Cryptology-ASIACRYPT 2013: 19th International Conference on the Theory and Application of Cryptology and Information Security, Bengaluru, India, December 1–5, 2013, Proceedings, Part II 19*. Springer. 2013, pp. 280–300 (cit. on p. 5).
- [BZ17] Dan Boneh and Mark Zhandry. “Multiparty key exchange, efficient traitor tracing, and more from indistinguishability obfuscation”. In: *Algorithmica* 79 (2017), pp. 1233–1285 (cit. on p. 5).

- [CG24] Andrea Coladangelo and Sam Gunn. “How to use quantum indistinguishability obfuscation”. In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*. 2024, pp. 1003–1008 (cit. on p. 13).
- [CHJV14] Ran Canetti, Justin Holmgren, Abhishek Jain, and Vinod Vaikuntanathan. “Indistinguishability obfuscation of iterated circuits and RAM programs”. In: *Cryptology ePrint Archive* (2014) (cit. on p. 5).
- [CHV23] Céline Chevalier, Paul Hermouet, and Quoc-Huy Vu. “Semi-quantum Copy-Protection and More”. In: *Theory of Cryptography: 21st International Conference, TCC 2023, Taipei, Taiwan, November 29–December 2, 2023, Proceedings, Part IV*. Taipei, Taiwan: Springer-Verlag, 2023, pp. 155–182. ISBN: 978-3-031-48623-4. DOI: 10.1007/978-3-031-48624-1_6. URL: https://doi.org/10.1007/978-3-031-48624-1_6 (cit. on pp. 6, 8, 10, 11, 17).
- [CLLZ21] Andrea Coladangelo, Jiahui Liu, Qipeng Liu, and Mark Zhandry. “Hidden Cosets and Applications to Unclonable Cryptography”. In: *Advances in Cryptology – CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2021, pp. 556–584. ISBN: 978-3-030-84241-3. DOI: 10.1007/978-3-030-84242-0_20. URL: https://doi.org/10.1007/978-3-030-84242-0_20 (cit. on pp. 6, 8, 11, 13, 17).
- [DQV+21] Lalita Devadas, Willy Quach, Vinod Vaikuntanathan, Hoeteck Wee, and Daniel Wichs. “Succinct LWE Sampling, Random Polynomials, and Obfuscation”. In: *Theory of Cryptography: 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8–11, 2021, Proceedings, Part II*. Raleigh, NC, USA: Springer-Verlag, 2021, pp. 256–287. ISBN: 978-3-030-90452-4. DOI: 10.1007/978-3-030-90453-1_9. URL: https://doi.org/10.1007/978-3-030-90453-1_9 (cit. on p. 15).
- [GGH+13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. “Candidate Indistinguishability Obfuscation and Functional Encryption for all Circuits”. In: *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*. 2013, pp. 40–49. DOI: 10.1109/FOCS.2013.13 (cit. on p. 15).
- [GIS+10] Vipul Goyal, Yuval Ishai, Amit Sahai, Ramarathnam Venkatesan, and Akshay Wadia. “Founding cryptography on tamper-proof hardware tokens”. In: *Theory of Cryptography: 7th Theory of Cryptography Conference, TCC 2010, Zurich, Switzerland, February 9–11, 2010. Proceedings 7*. Springer. 2010, pp. 308–326 (cit. on p. 5).
- [GKR08] Shafi Goldwasser, Yael Tauman Kalai, and Guy N Rothblum. “One-time programs”. In: *Advances in Cryptology–CRYPTO 2008: 28th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2008. Proceedings 28*. Springer. 2008, pp. 39–56 (cit. on p. 4).
- [GLR+24] Aparna Gupte, Jiahui Liu, Justin Raizes, Bhaskar Roberts, and Vinod Vaikuntanathan. *Quantum One-Time Programs, Revisited*. 2024. arXiv: 2411.01876 [cs.CR]. URL: <https://arxiv.org/abs/2411.01876> (cit. on pp. 4, 6, 7, 10, 18).
- [GM24] Sam Gunn and Ramis Movassagh. “Quantum one-time protection of any randomized algorithm”. In: *arXiv preprint arXiv:2411.03305* (2024) (cit. on p. 4).

- [HKKW19] Dennis Hofheinz, Akshay Kamath, Venkata Koppula, and Brent Waters. “Adaptively secure constrained pseudorandom functions”. In: *International Conference on Financial Cryptography and Data Security*. Springer. 2019, pp. 357–376 (cit. on p. 5).
- [KHF+20] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, et al. “Spectre attacks: Exploiting speculative execution”. In: *Communications of the ACM* 63.7 (2020), pp. 93–101 (cit. on p. 4).
- [KPTZ13] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. “Delegatable pseudorandom functions and applications”. In: *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 2013, pp. 669–684 (cit. on p. 5).
- [KSS21] Daniel M Kane, Shahed Sharif, and Alice Silverberg. “Quantum money from quaternion algebras”. In: *arXiv preprint arXiv:2109.12643* (2021) (cit. on p. 4).
- [LMZ23] Jiahui Liu, Hart Montgomery, and Mark Zhandry. “Another round of breaking and making quantum money: How to not build it from lattices, and more”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2023, pp. 611–638 (cit. on p. 4).
- [NC11] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. 10th. USA: Cambridge University Press, 2011. ISBN: 1107002176 (cit. on p. 16).
- [PS18] Chris Peikert and Sina Shiehian. “Privately constraining and programming PRFs, the LWE way”. In: *IACR International Workshop on Public Key Cryptography*. Springer. 2018, pp. 675–701 (cit. on pp. 5, 6).
- [Wat15] Brent Waters. “A punctured programming approach to adaptively secure functional encryption”. In: *Annual Cryptology Conference*. Springer. 2015, pp. 678–697 (cit. on p. 5).
- [Wie83] Stephen Wiesner. “Conjugate Coding”. In: *SIGACT News* 15.1 (Jan. 1983), pp. 78–88. ISSN: 0163-5700. DOI: 10.1145/1008908.1008920. URL: <https://doi.org/10.1145/1008908.1008920> (cit. on p. 4).
- [Zha21] Mark Zhandry. “Quantum lightning never strikes the same state twice. or: quantum money from cryptographic assumptions”. In: *Journal of Cryptology* 34 (2021), pp. 1–56 (cit. on pp. 4, 13, 15).
- [Zha23] Mark Zhandry. “Quantum money from abelian group actions”. In: *arXiv preprint arXiv:2307.12120* (2023) (cit. on p. 4).